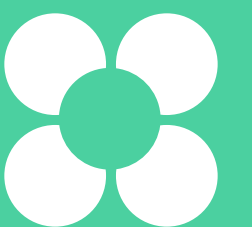


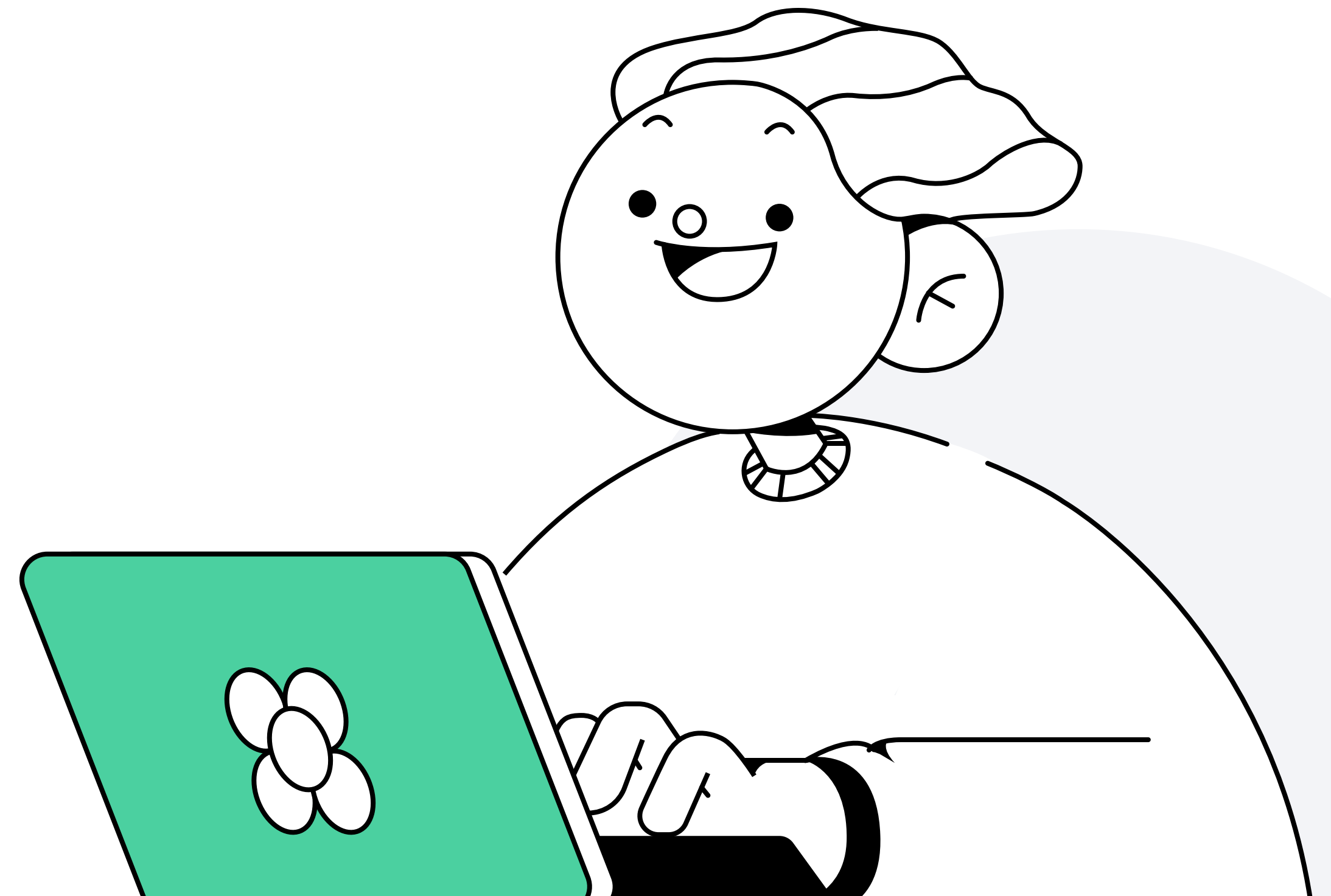
# Микросервисы: принципы

Михаил Триполитов  
Банк Открытие, архитектор решений

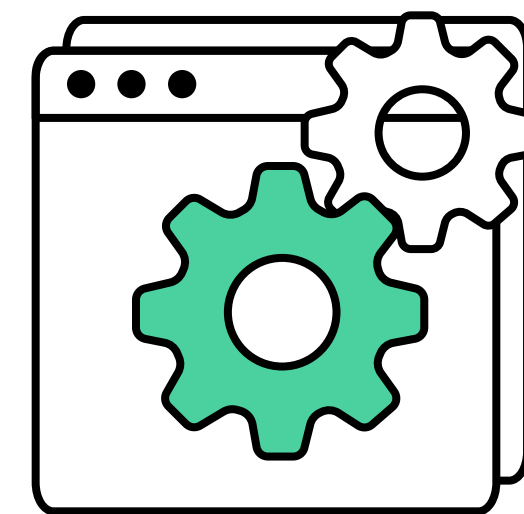


# План занятия

- 1 Проектирование системы
- 2 Разделение на сервисы
- 3 Взаимодействие между сервисами
- 4 Двенадцать факторов



# Проектирование системы



Михаил Триполитов  
Банк Открытие, архитектор решений



# Тема занятия

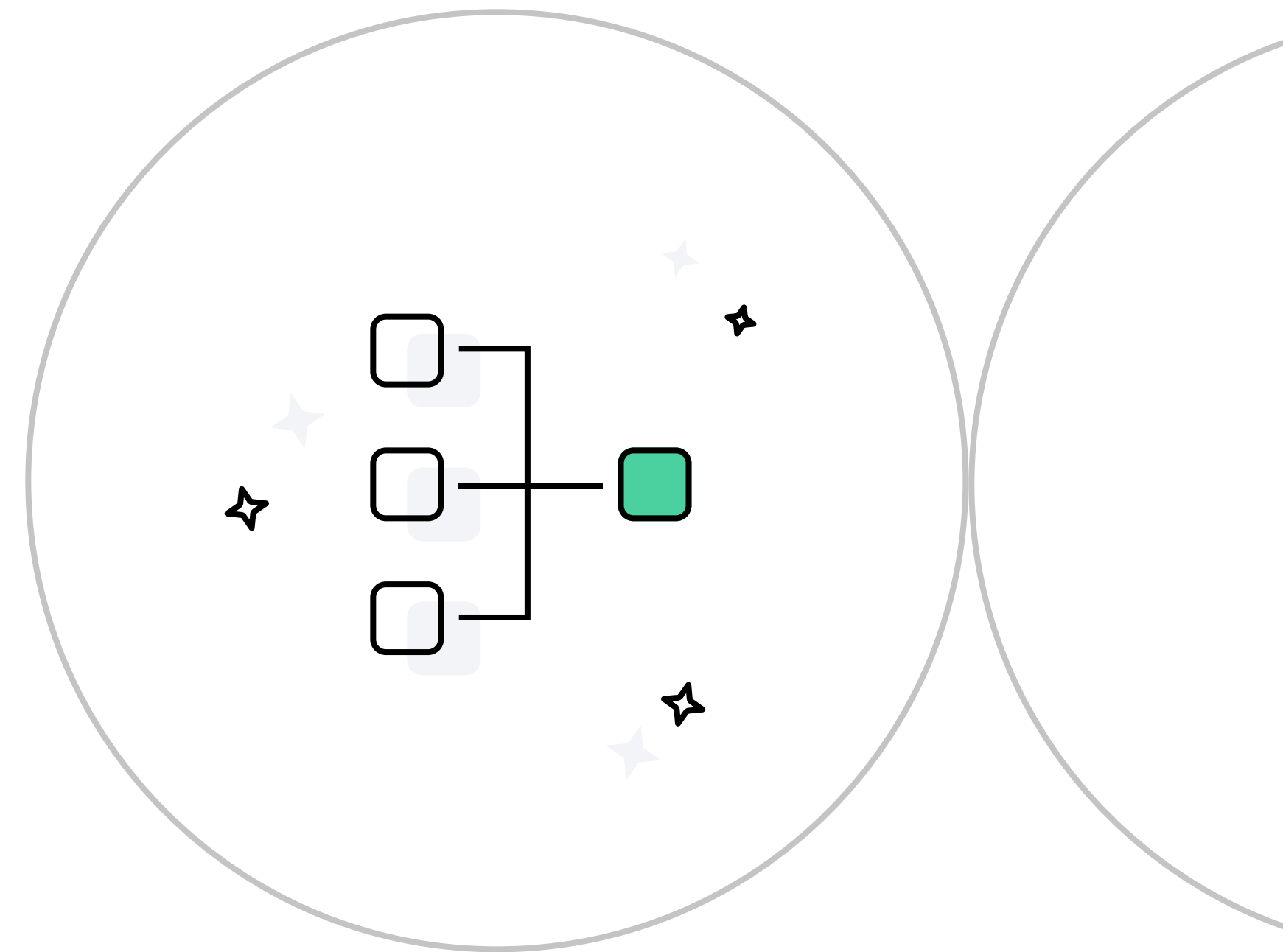
- 1 Подход к проектированию
- 2 Цели, принципы и практики
- 3 Необходимый минимум базовых практик



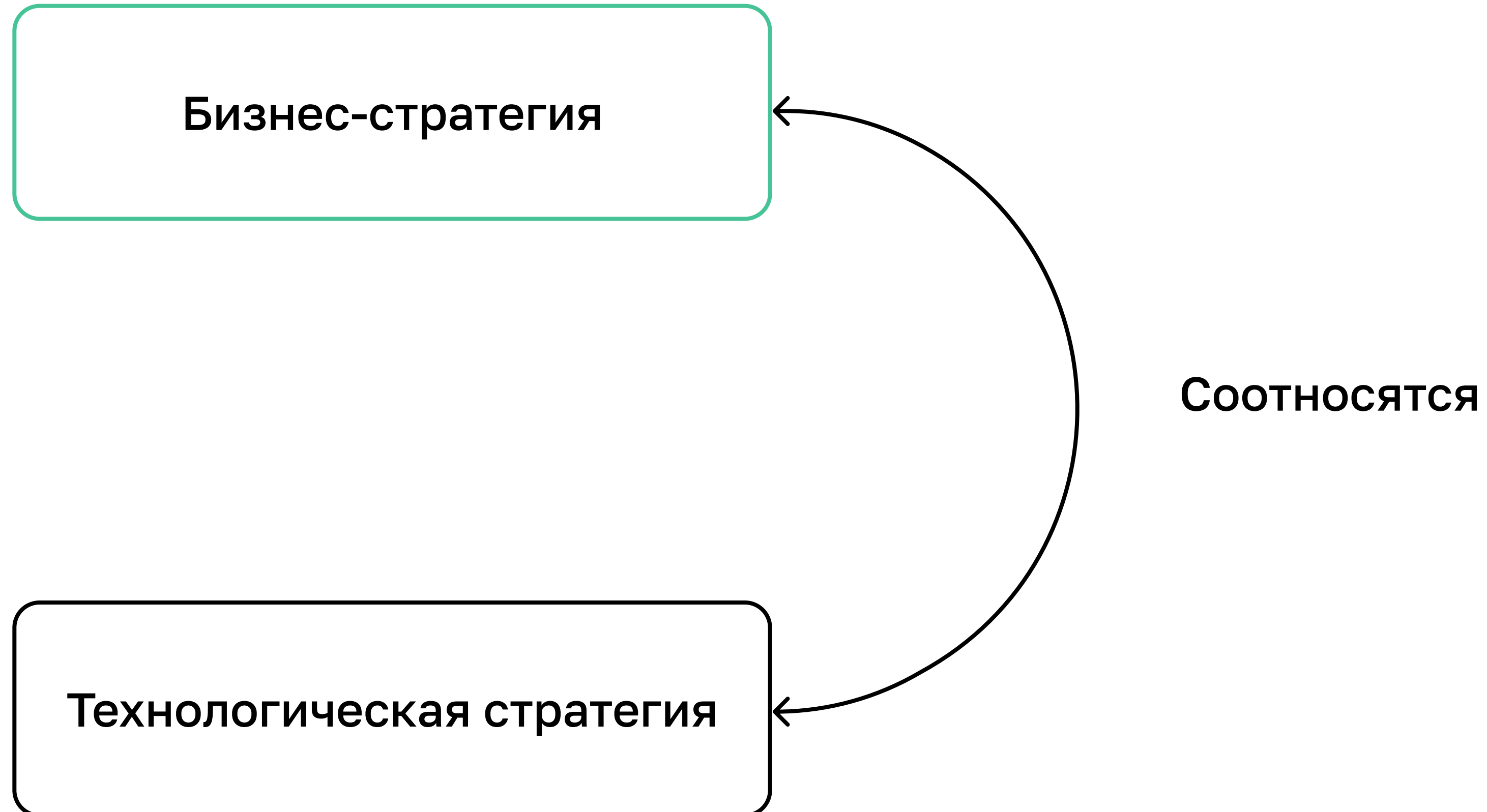


# Архитектура

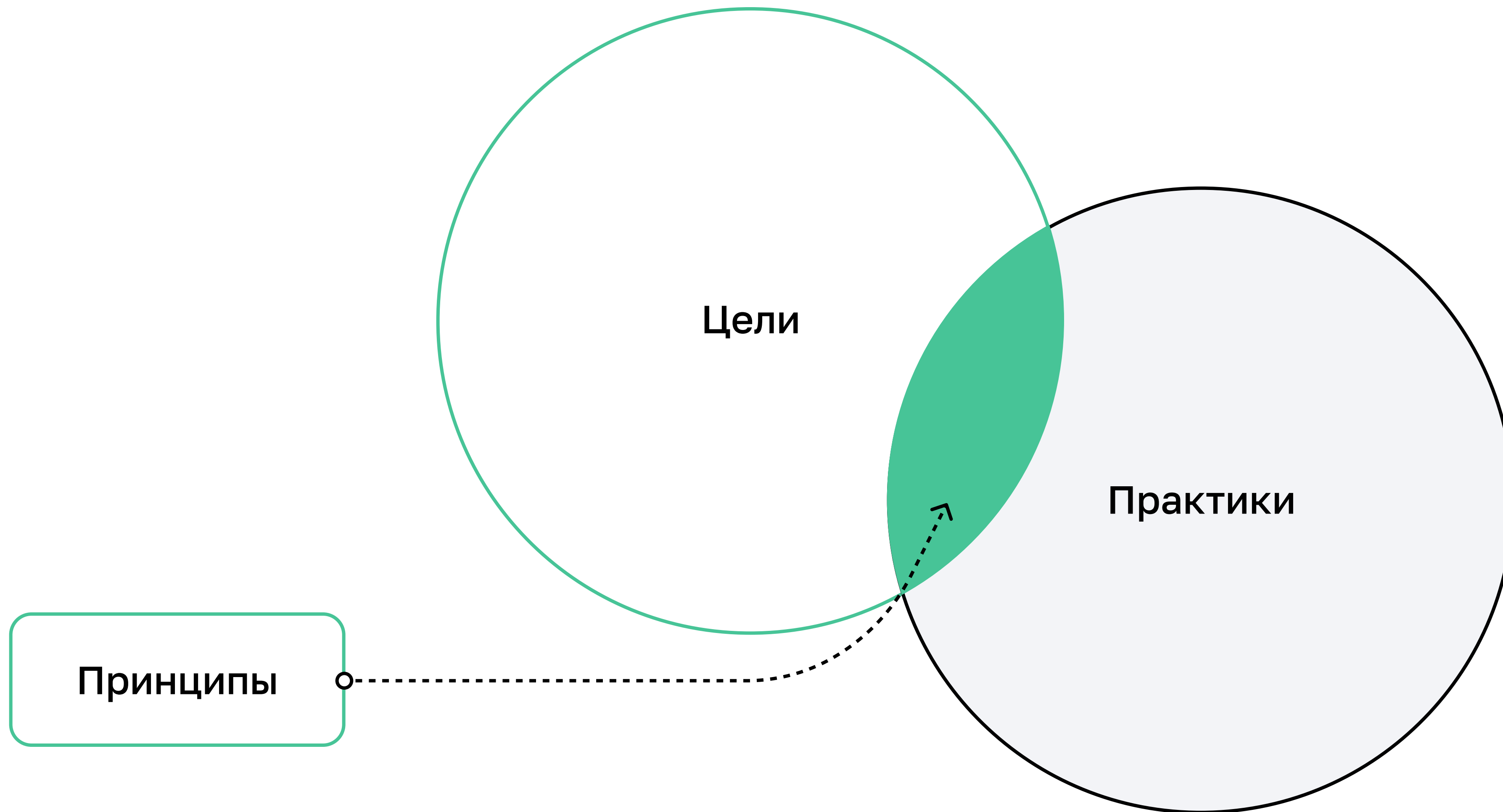
- ✗ Детальный план
- ✗ Подробная документация
- ✓ Правила
- ✓ Принципы
- ✓ Практики
- ✓ Ограничения



# Стратегические цели



# Принципы





# Практики

## Цели

- Масштабирование бизнеса
- Сокращение используемого операторами программного обеспечения
- Оптимизация ресурсов и затрат

## Принципы

- Обеспечивать согласованные интерфейсы и потоки данных
- Выбирать решения с быстрым циклом обратной связи
- Уменьшать избыточную сложность, заменяя дублирующие системы

## Практики

- Использовать HTTP для межсервисных интеграций
- Исключать интеграционные базы данных
- Использовать независимые сервисы
- Применять непрерывную интеграцию и постоянное развёртывание
- Использовать мониторинг для получения состояния сервисов
- Автоматически генерировать клиентские библиотеки для всех публикуемых интерфейсов

# Практики: необходимым минимум

- Централизованный мониторинг
- Централизованный сбор логов
- Ограниченный набор допустимых интерфейсов
- Безопасное поведение



# Итоги

- ❗ Максимум контроля на то, что происходит между сервисами, и свобода того, что внутри
- Управляемость и устойчивость системы важнее независимости сервисов





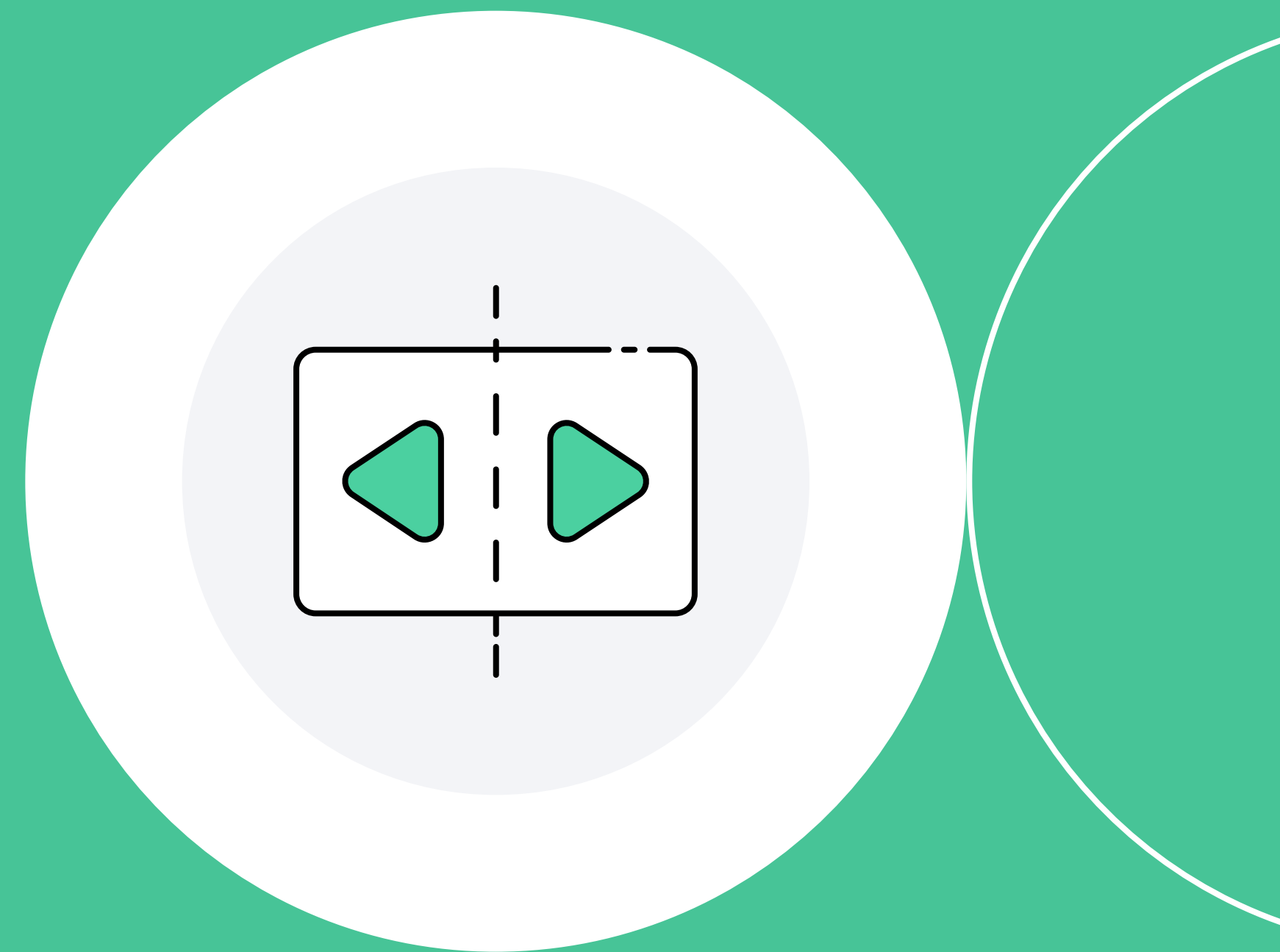
# Итоги

- Максимум контроля на то, что происходит между сервисами, и свобода того, что внутри
- ! Управляемость и устойчивость системы важнее независимости сервисов



# Разделение на сервисы

Михаил Триполитов  
Банк Открытие, архитектор решений





# Тема занятия

- 1 Признаки хорошего сервиса
- 2 Определение границ ответственности сервиса
- 3 Причины уменьшать сервисы



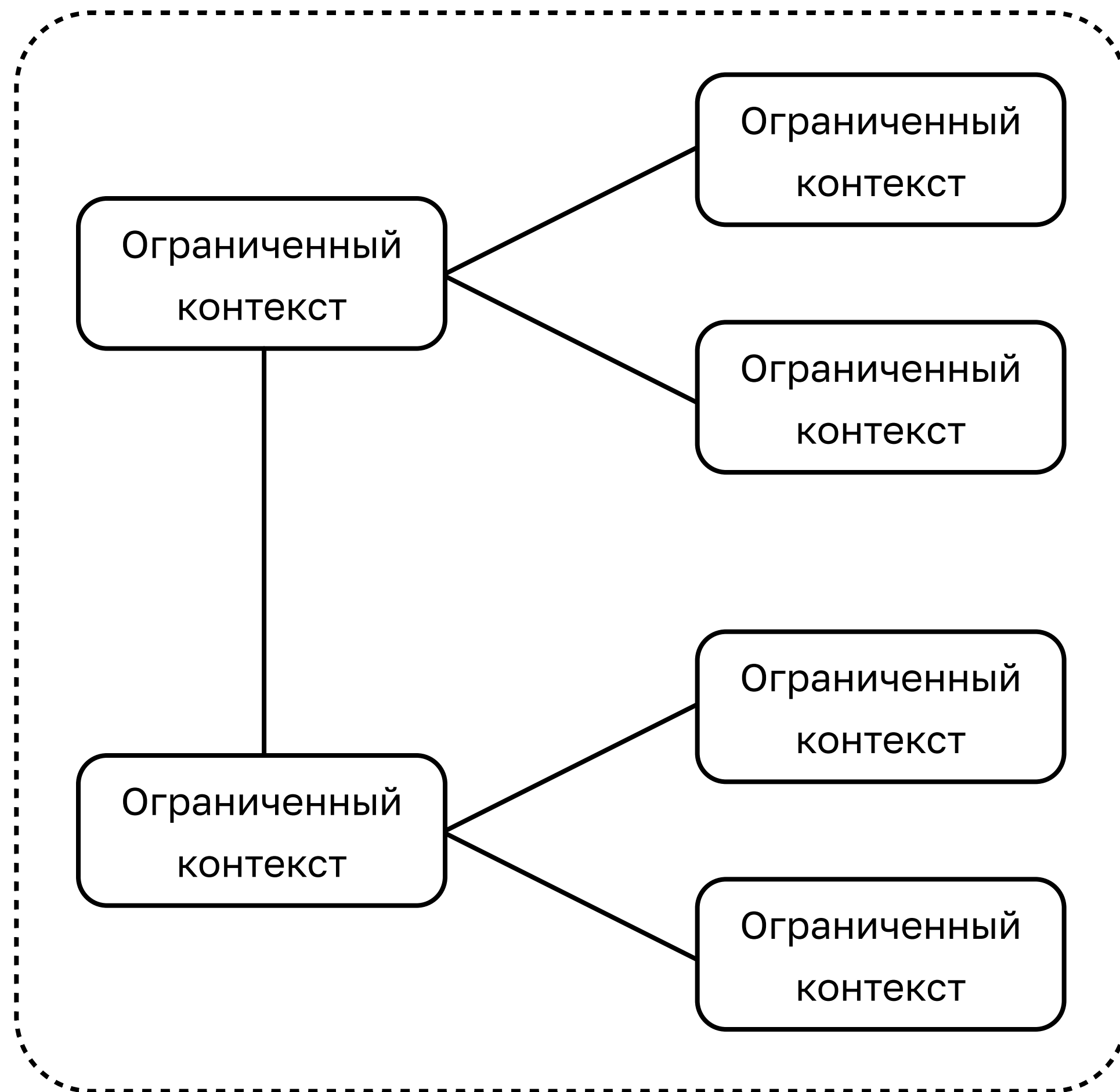
# Признаки хорошего сервиса

- **Loose Coupling**  
Минимизировать влияние изменений в одном сервисе на всю систему
- **High Cohesion**  
Минимизировать необходимость менять несколько сервисов при изменении поведения системы



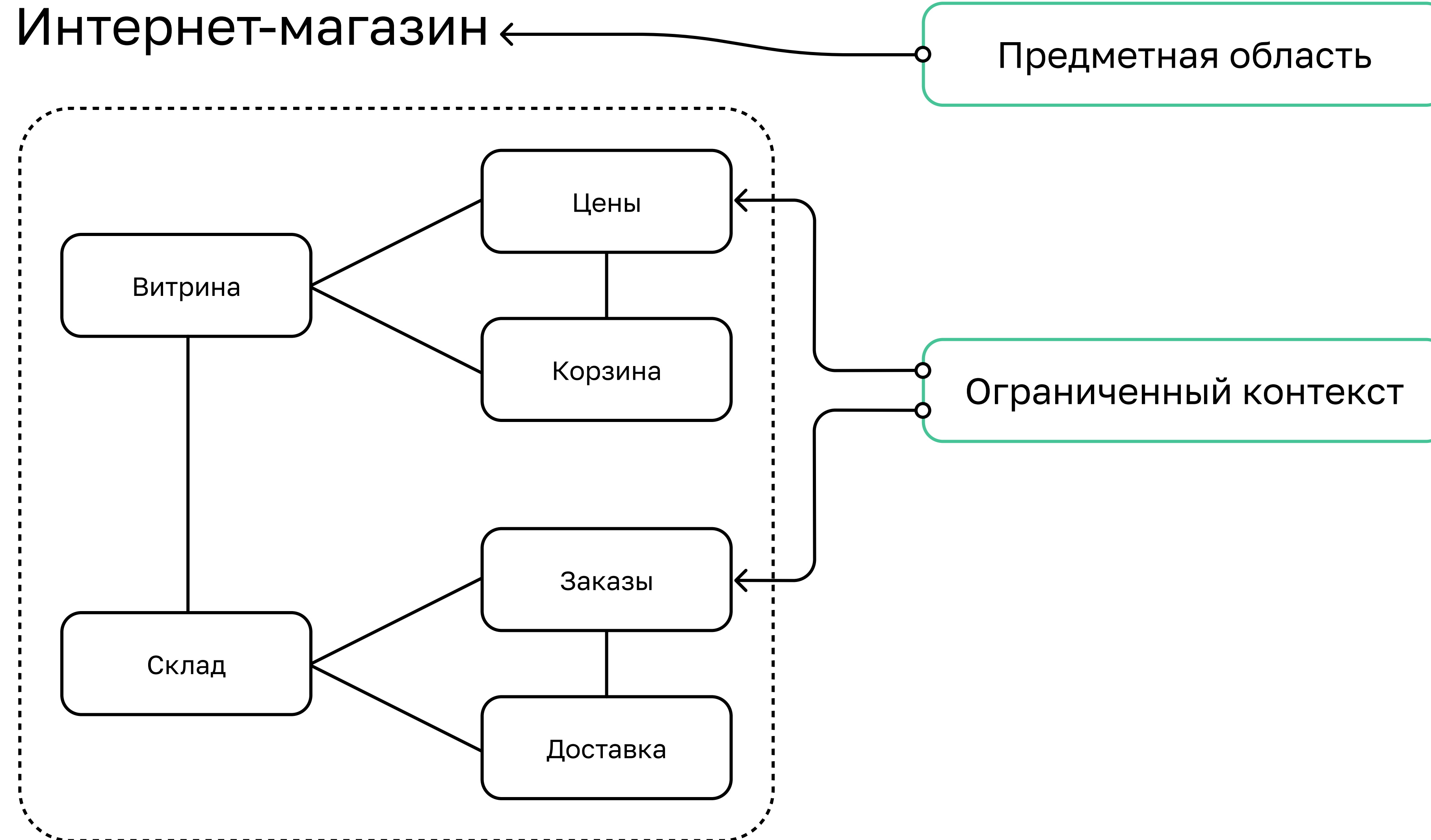
# Bounded Context

Предметная область





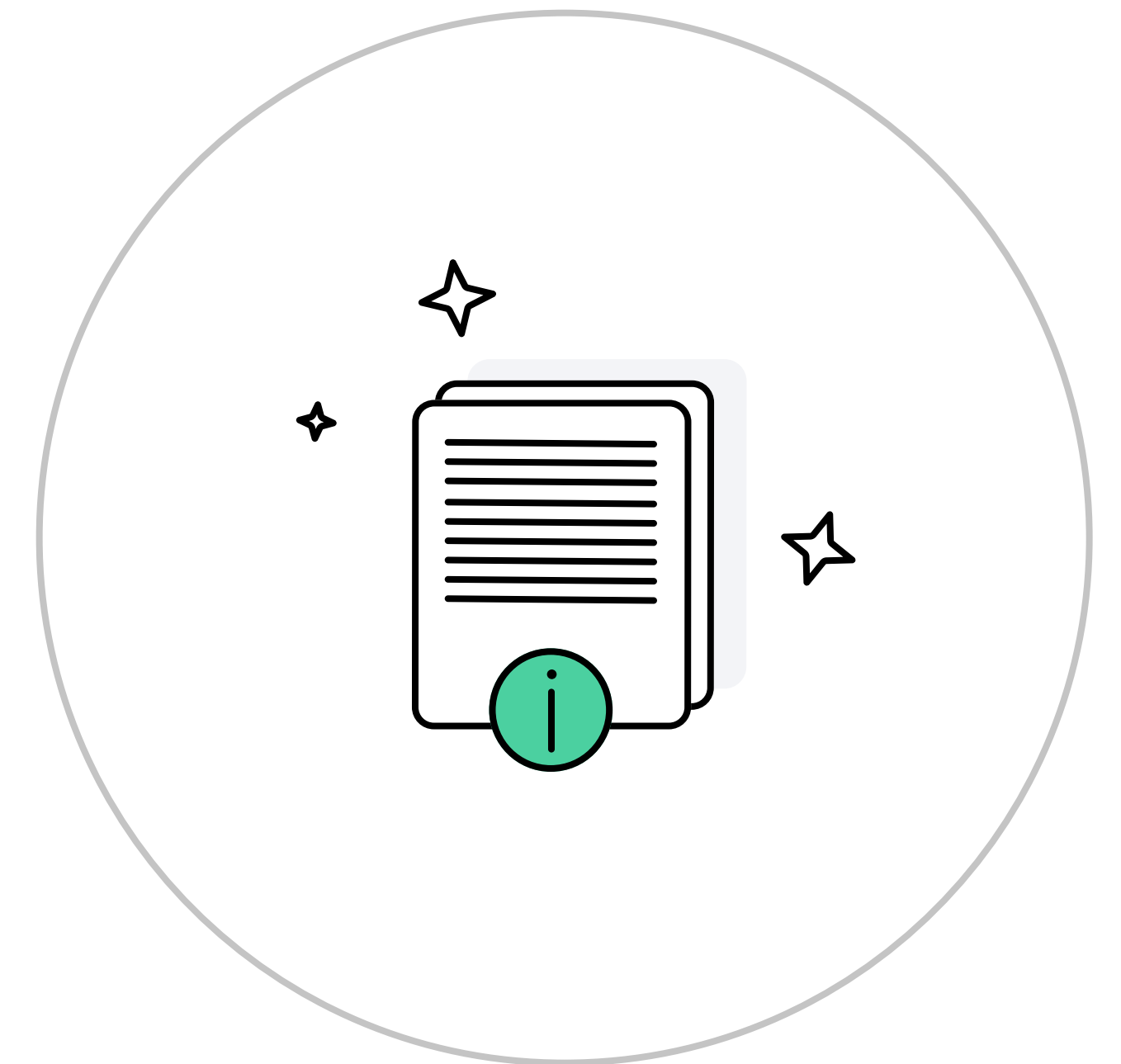
# Bounded Context



# Один контекст. Один сервис. Одна команда

Ограниченный контекст — это:

- отдельная команда
- отдельный репозиторий
- отдельная схема базы данных
- отдельная процедура тестирования
- отдельная процедура выкладки



# Причины уменьшать

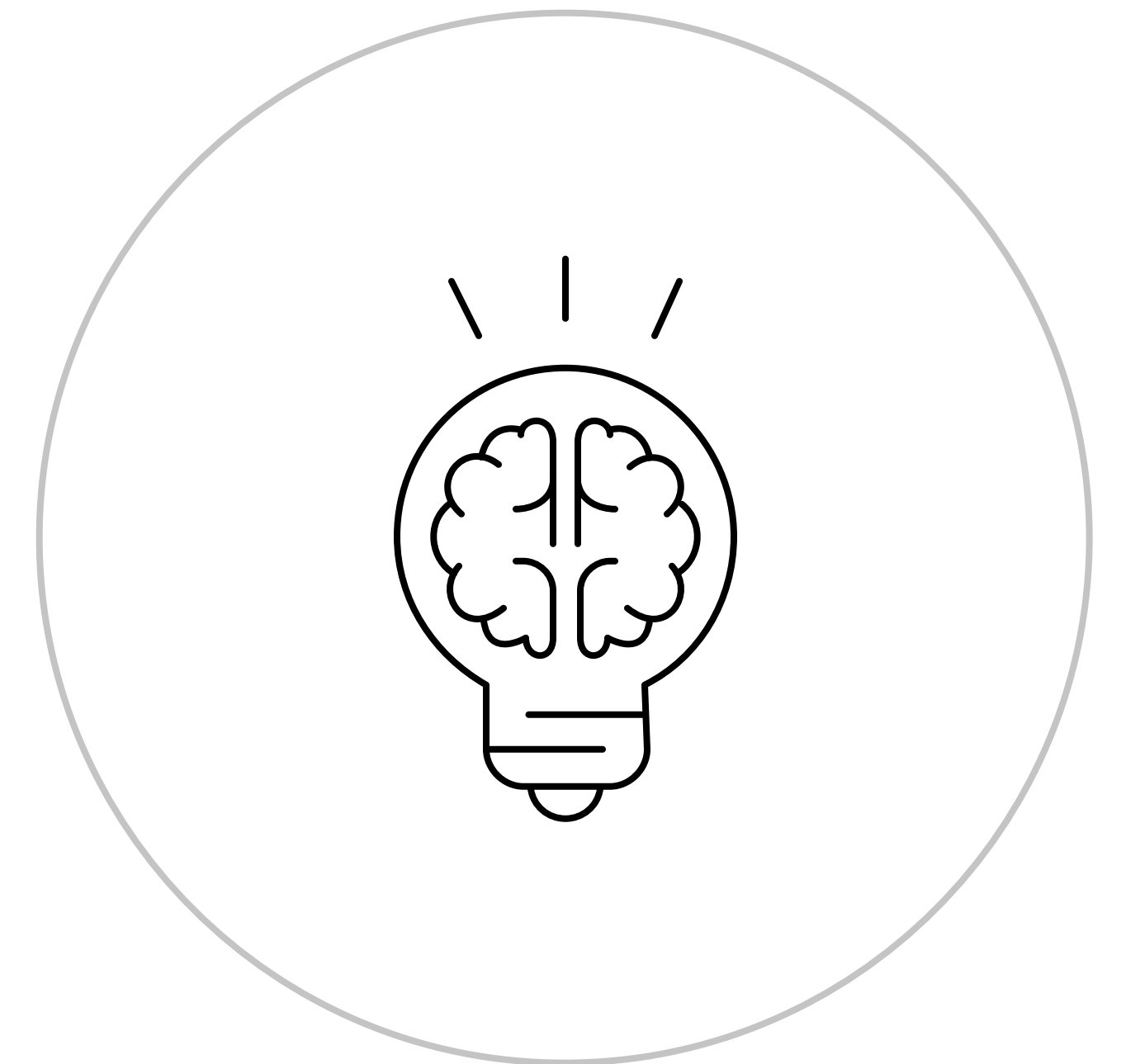
- Частота изменений
- Масштабирование
- Независимость
- Сложность





# Bounded Context

- ❗ Использование ограниченных контекстов для разбиения системы на сервисы позволит **обеспечить слабую связность и сильное зацепление**, соблюдая баланс размера сервисов



# Итоги

- ❗ Хороший сервис должен обладать слабой связностью и сильным зацеплением
- Применение предметно-ориентированного проектирования ограниченных контекстов для разбиения системы на сервисы позволит обеспечить слабую связность и сильное зацепление



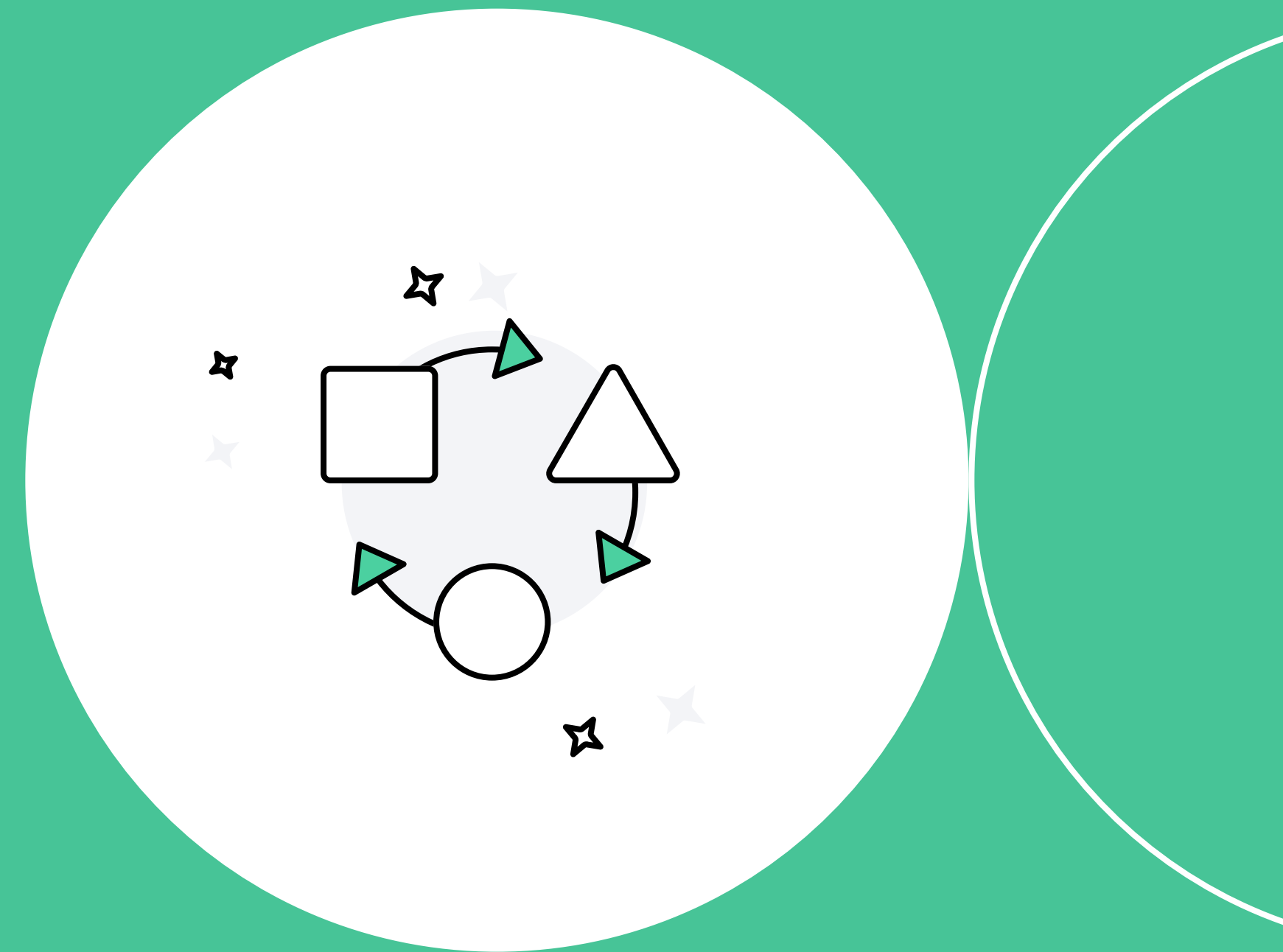


# Итоги

- Хороший сервис должен обладать слабой связностью и сильным зацеплением
- ! Применение предметно-ориентированного проектирования ограниченных контекстов для разбиения системы на сервисы позволит обеспечить слабую связность и сильное зацепление



# Взаимодействие между сервисами



Михаил Триполитов  
Банк Открытие, архитектор решений

# Тема занятия

- 1 Выбор протокола межсервисного взаимодействия
- 2 Варианты межсервисного взаимодействия
- 3 Протоколы межсервисного взаимодействия
- 4 Версионирование контрактов





# Выбор

## Какой протокол выбрать?

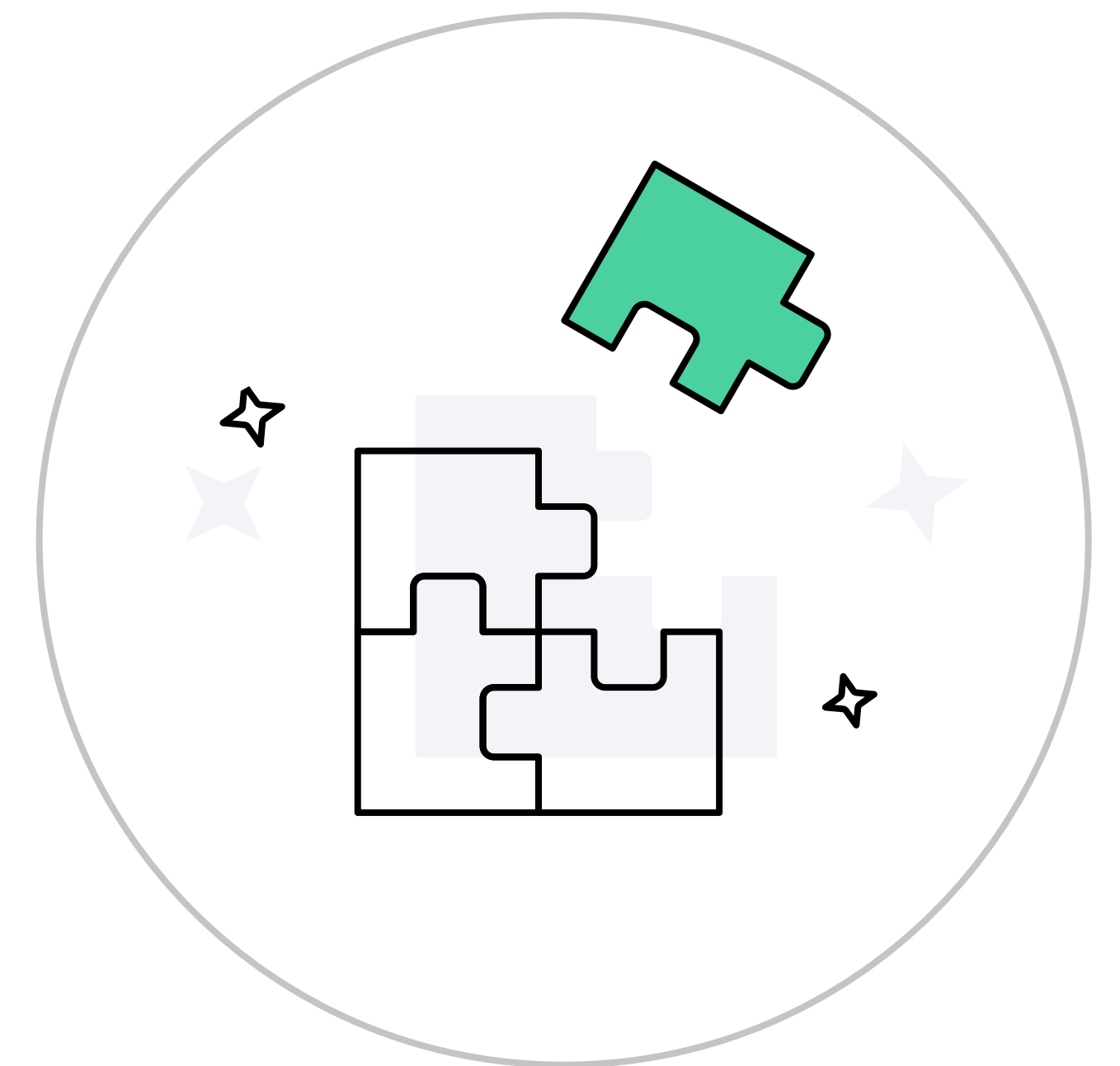
- XML-RPC
- JSON-RPC
- gRPC
- REST
- GraphQL
- SOAP
- ...

## Что ещё выбрать?

- Синхронно/Асинхронно
- Оркестрация/Хореография
- RPC / Команды и события
- ...

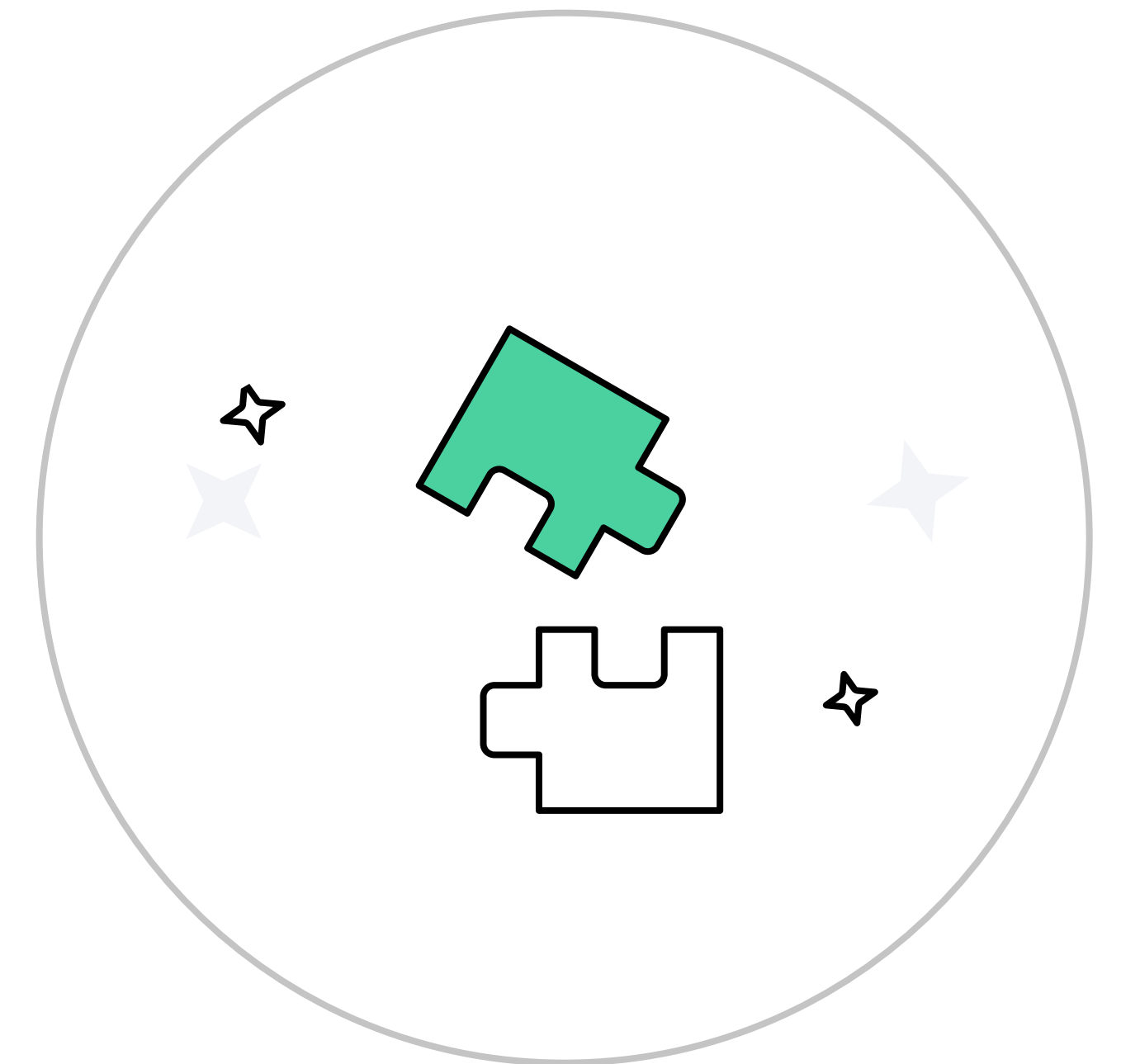
# Выбор

- Обратная совместимость
- Технологическая независимость
- Забота о потребителях
- Чёткий интерфейс



# Обратная совместимость

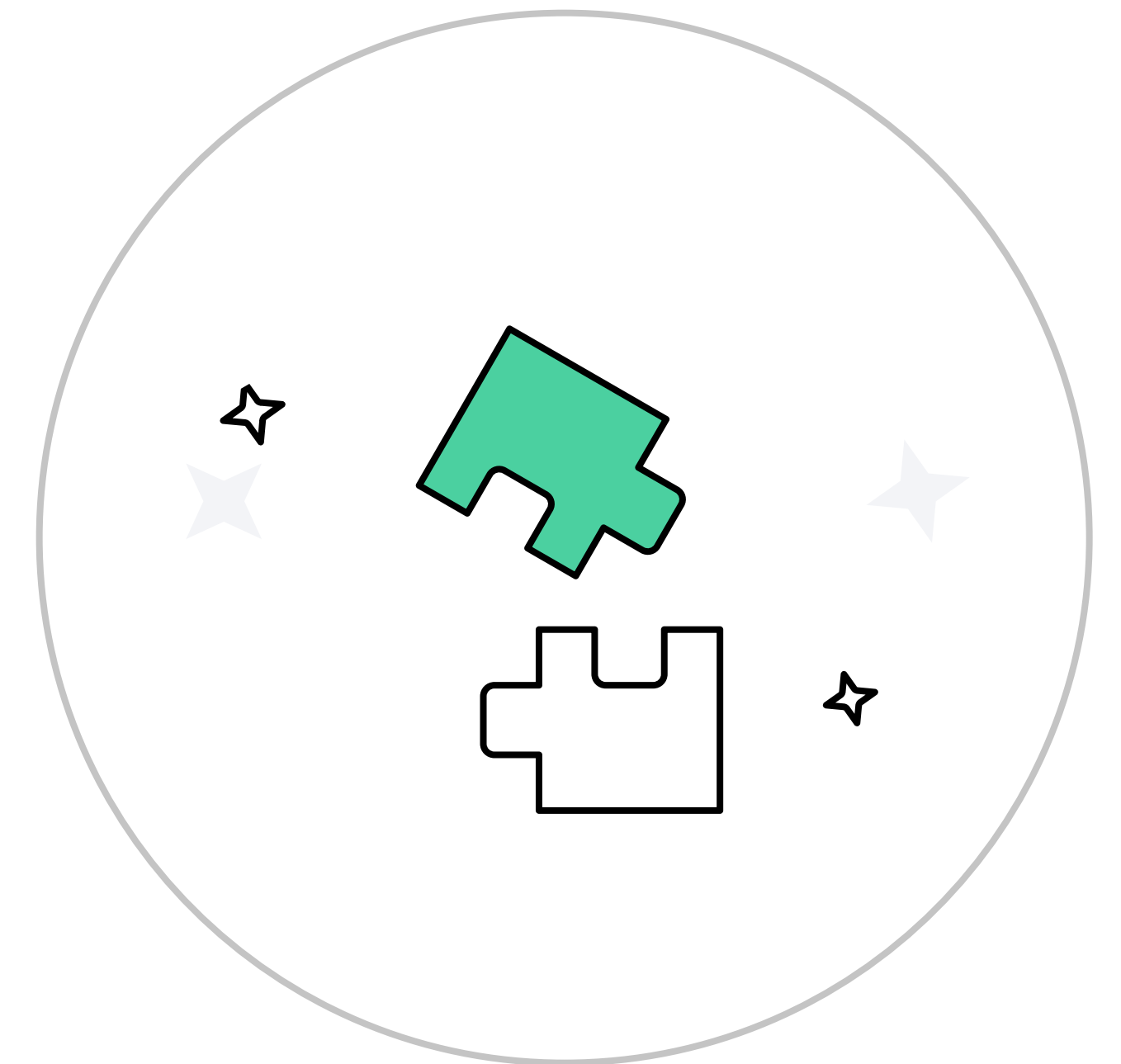
! Избегайте обратно несовместимых изменений





# Технологическая независимость

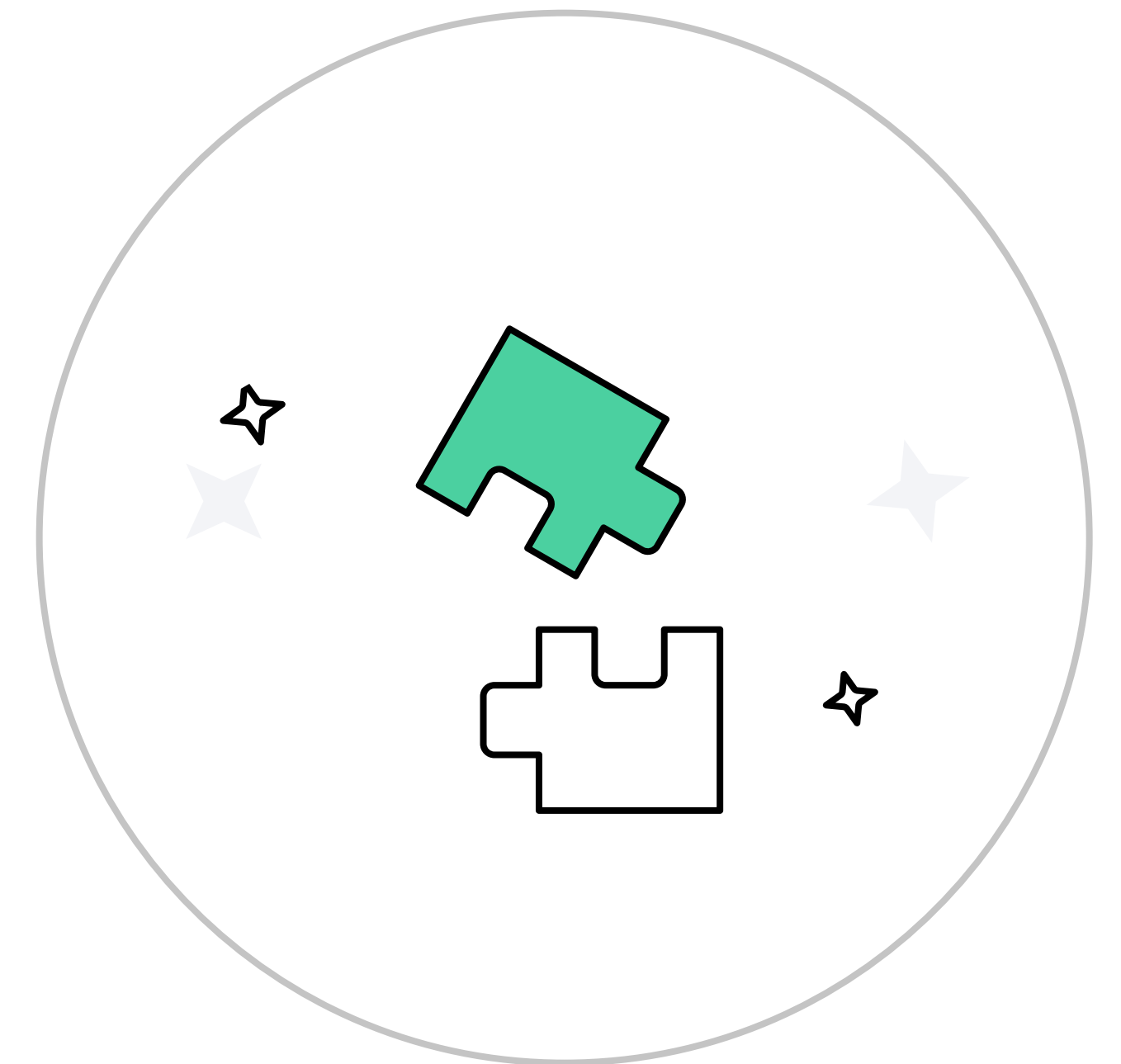
- ❗ Избегайте интеграционных технологий и подходов, которые привязаны к какой-то конкретной технологической платформе



# Забота о потребителях

❗ Стремитесь снизить требования к клиентам и упростить использование интерфейсов:

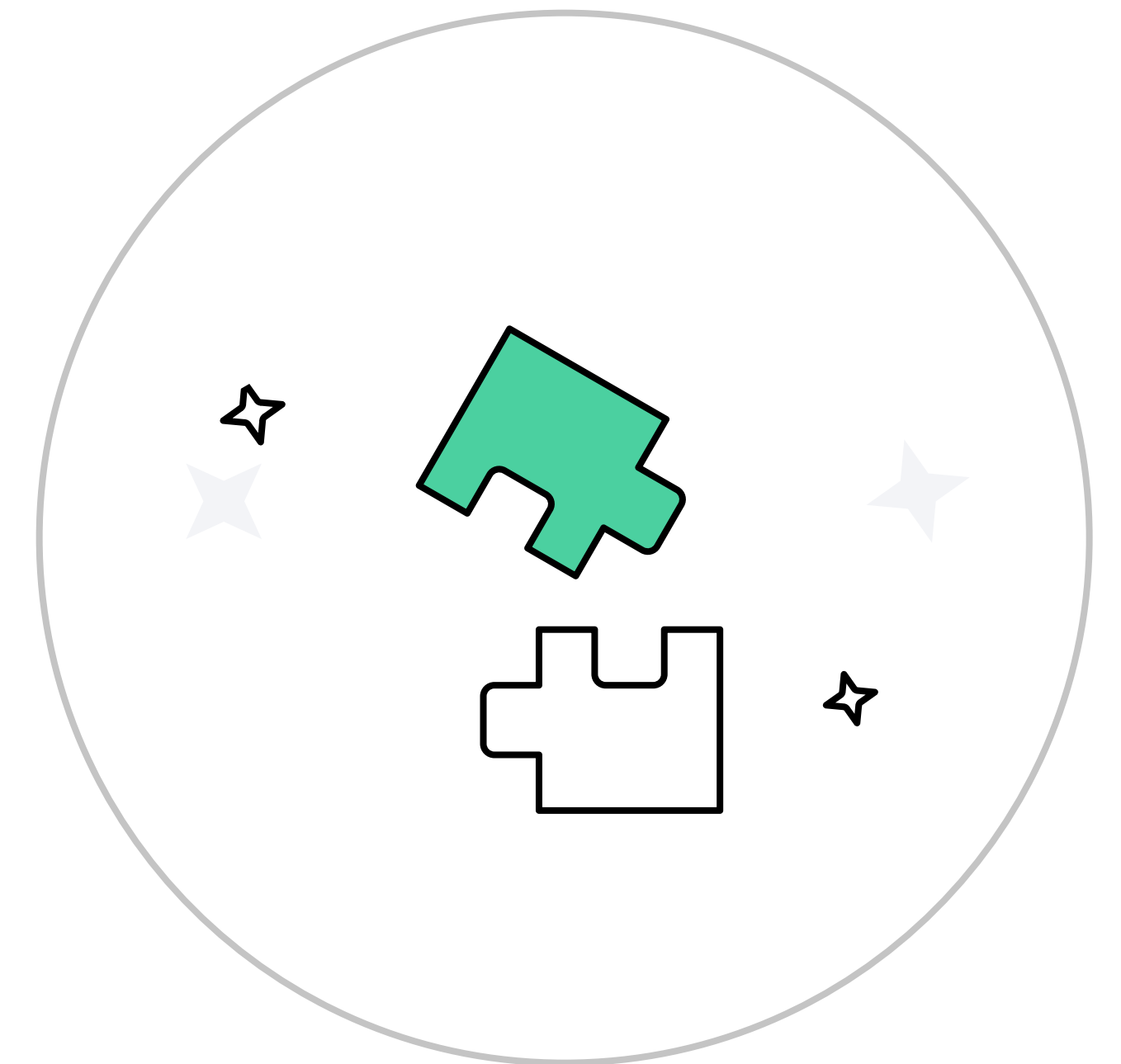
- документация
- простое и понятное API
- готовые клиентские библиотеки





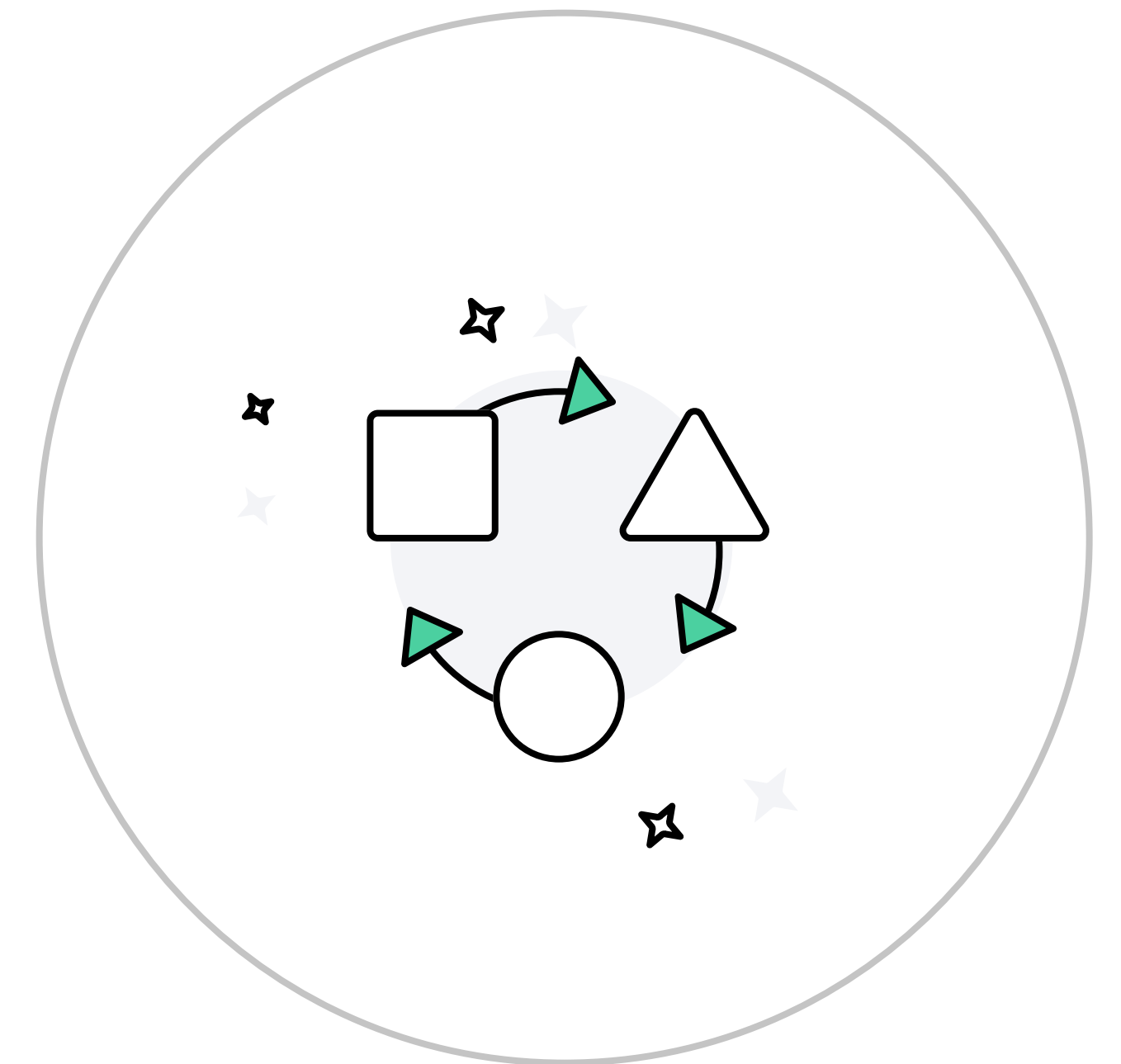
# Чёткий интерфейс

- ❗ Прячьте детали реализации от потребителей интерфейсов. Избегайте интеграционных подходов и технологий, которые раскрывают детали внутренней реализации потребителям

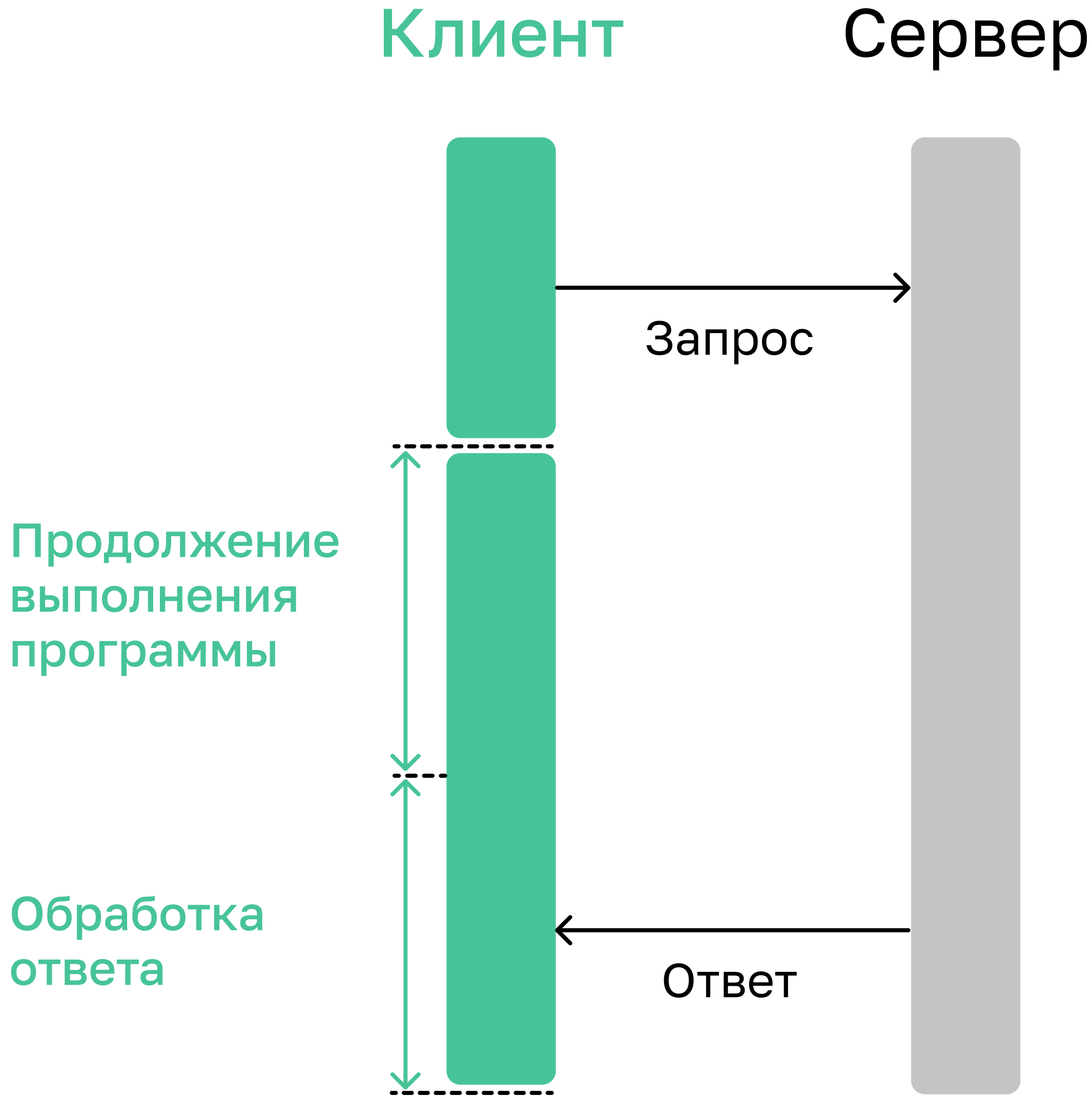
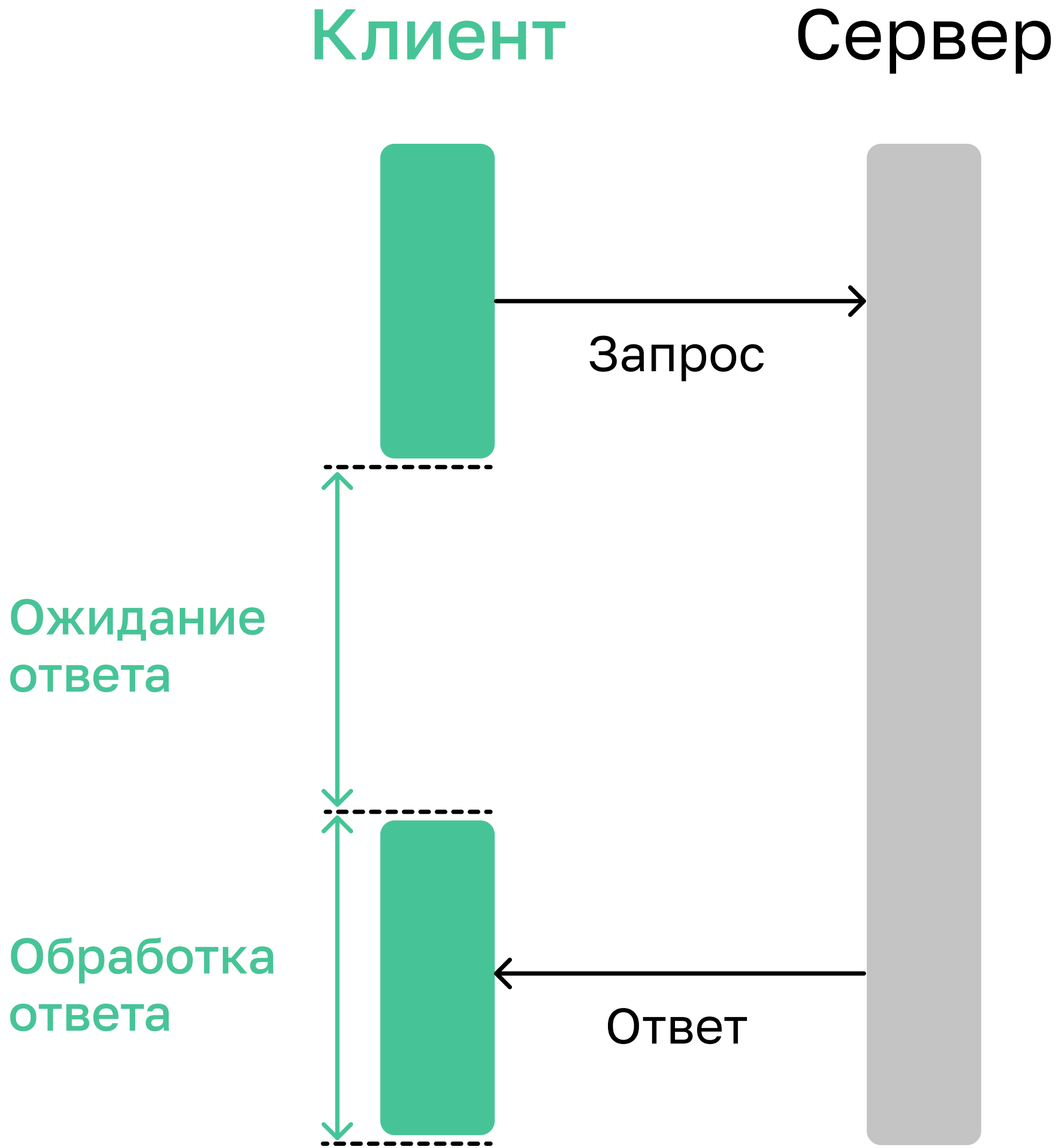


# Варианты межсервисного взаимодействия

- ! Синхронное или асинхронное
- ! Запрос-ответ или событийная модель
- ! Оркестрация или хореография



# Синхронное или асинхронное взаимодействие





# Запрос-ответ или события

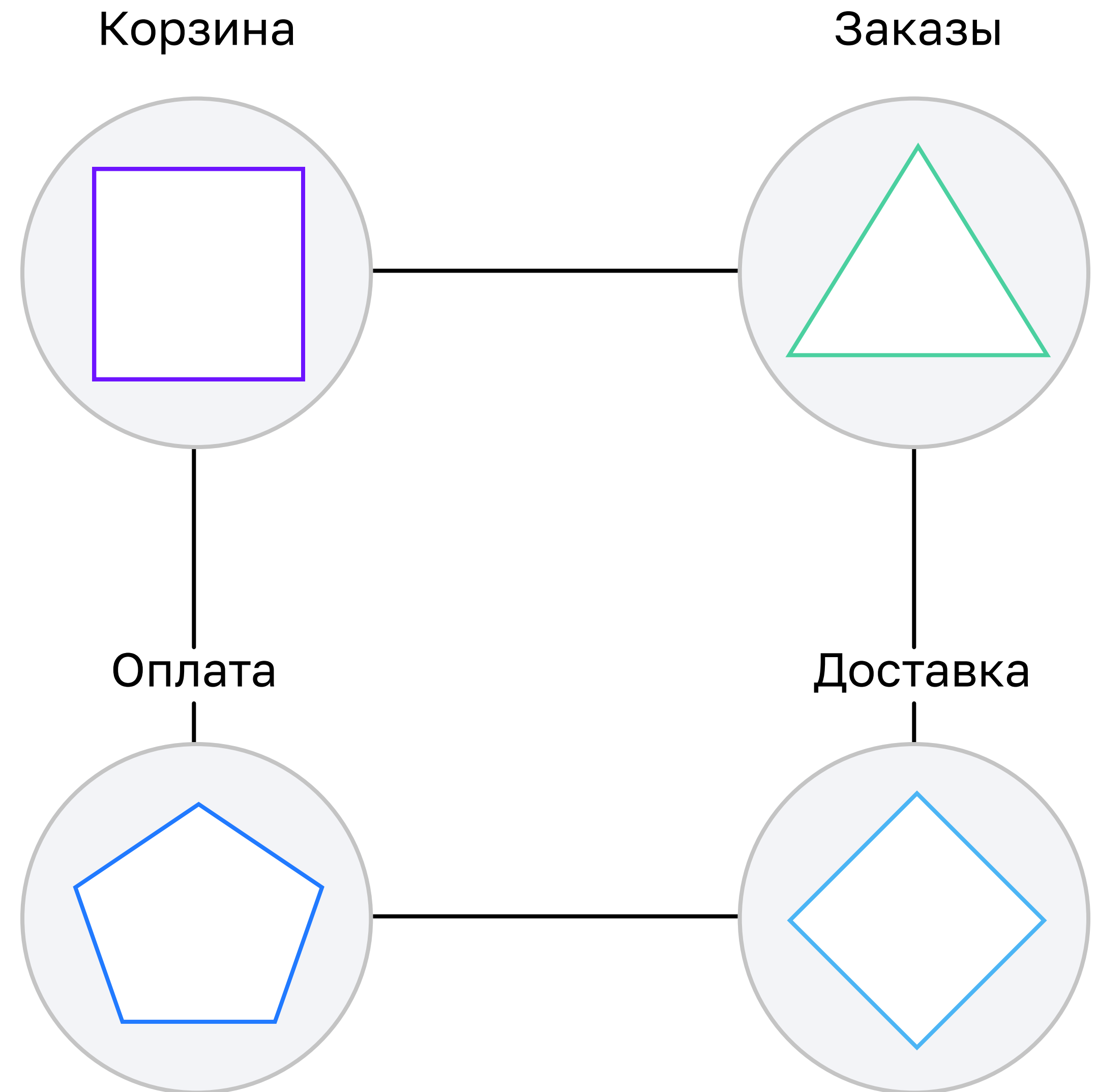
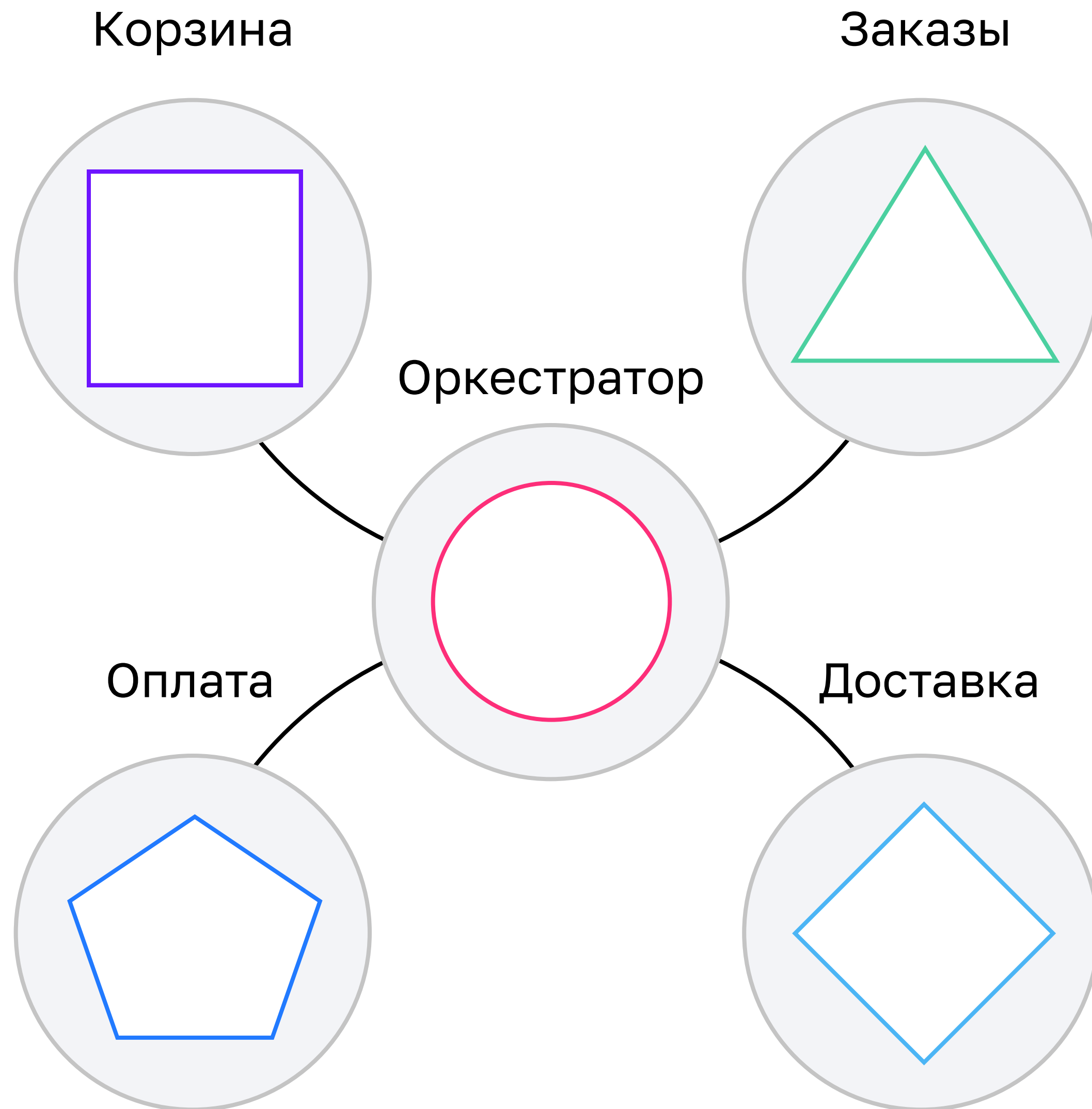
## Запрос-ответ

- ✓ Проще для понимания
- ✓ Синхронное и асинхронное взаимодействие
- ✗ Централизует бизнес-логику
- ✗ Увеличивает связность

## Событийная модель

- ✓ Уменьшает связность
- ✓ Децентрализация логики
- ✓ Простота расширения системы
- ✗ Общая сложность системы
- ✗ Только асинхронный подход

# Оркестрация или хореография



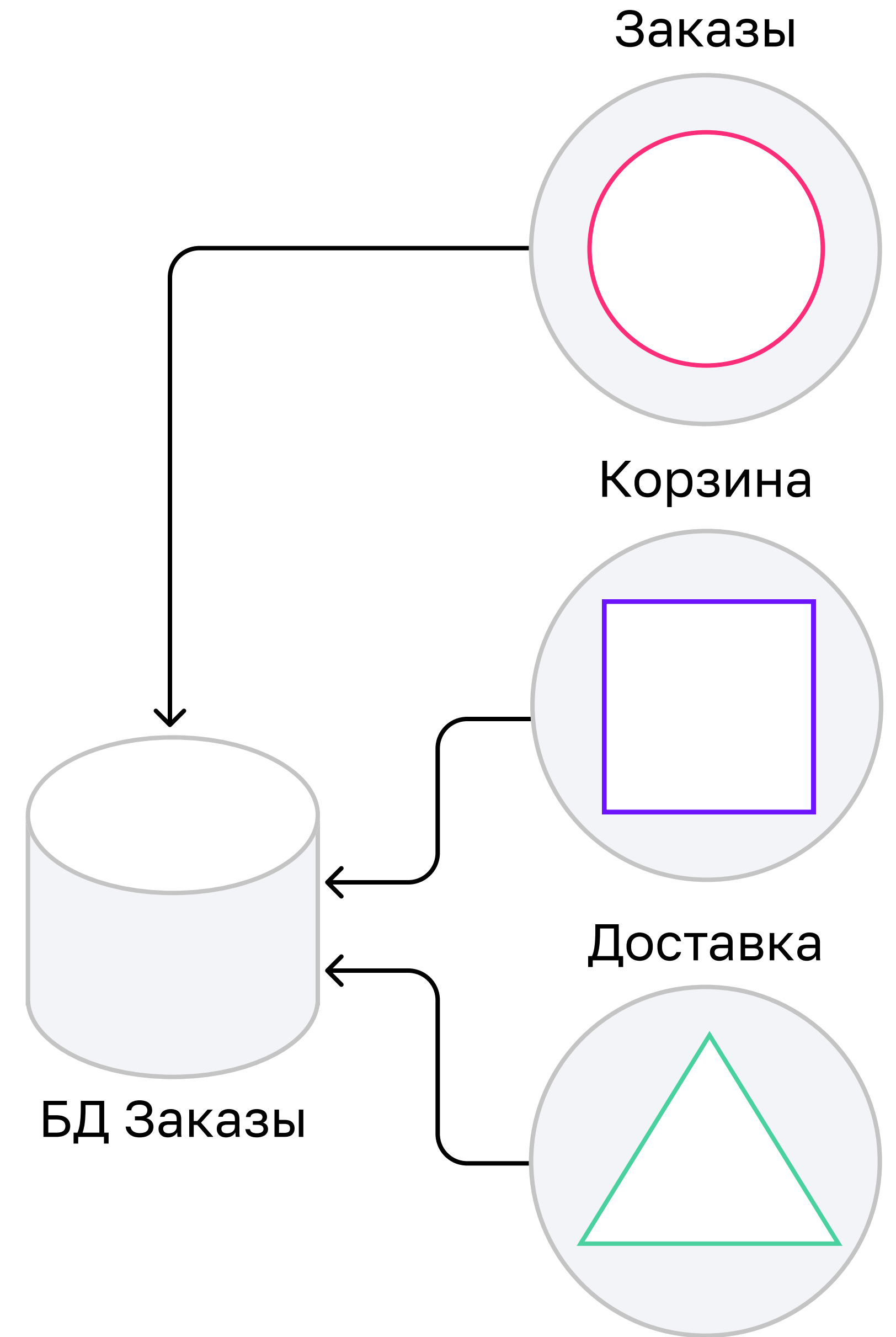
# Протоколы межсервисного взаимодействия

- ✗ Общая база данных
- ! Remote Procedure Call (RPC)
- ! REpresentation State Transfer (REST)
- ! GraphQL
- ! Событийная модель



# Общая база данных

- ✗ Детали реализации доступны другим сервисам
- ✗ Потребители ограничены выбранной технологией БД
- ✗ У данных нет единого владельца — логика по манипуляции данными распределена по разным сервисам



# Remote Procedure Calls

- ✓ Детали реализации недоступны другим сервисам
- ✓ При правильном выборе технологии потребители не ограничены одним стеком
- ✓ Простота использования
- ✗ Расширение моделей возможно только через добавление полей
- ✗ Ограниченная поддержка инфраструктурными инструментами



# REST – REpresentation State Transfer

- ✓ Детали реализации недоступны другим сервисам
- ✓ Потребители не ограничены одним стеком
- ✓ Простота использования
- ✓ Хорошая поддержка инфраструктурными инструментами
- ✓ Текстовый формат данных JSON, XML
- ✗ Расширение моделей возможно только через добавление полей
- ✗ Не всегда возможно описать модель в терминах протокола HTTP

# GraphQL

- ✓ Детали реализации недоступны другим сервисам
- ✓ Потребители не ограничены одним стеком
- ✓ Возможность получить несколько ресурсов одним запросом
- ✓ Возможность указать необходимые данные
- ✗ Расширение моделей возможно только через добавление полей
- ✗ Нет поддержки кеширования со стороны инфраструктуры

# Событийная модель

- ✓ Детали реализации недоступны другим сервисам
- ✓ Потребители не ограничены одним стеком
- ✓ Возможность получить несколько ресурсов одним запросом
- ✓ Низкая общая связность решения
- ✓ Высокая гибкость и способность к расширению
- ✗ Потребители ограничены выбранной технологией
- ✗ Высокая общая сложность системы
- ✗ Повышенные требования к инфраструктуре



# Итоги

- ☒ Не используйте интеграцию через общую базу данных
- ☐ Предпочитайте оркестрацию хореографии
- ☐ REST — отличный выбор для request/response интеграций
- ☐ Избегайте обратно несовместимых изменений и необходимости версионировать эндпоинты



# Итоги

- ☐ Не используйте интеграцию через общую базу данных
- ☒ ! Предпочитайте оркестрацию хореографии
- ☐ REST — отличный выбор для request/response интеграций
- ☐ Избегайте обратно несовместимых изменений и необходимости версионировать эндпоинты



# Итоги

- ☐ Не используйте интеграцию через общую базу данных
- ☐ Предпочитайте оркестрацию хореографии
- ☒ ! REST — отличный выбор для request/response интеграций
- ☐ Избегайте обратно несовместимых изменений и необходимости версионировать эндпоинты





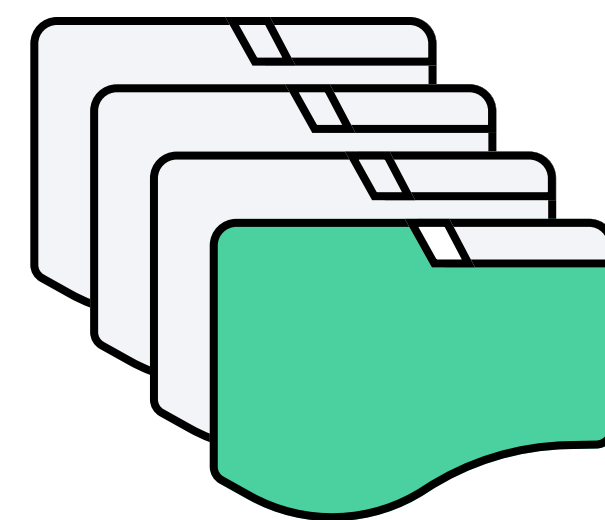
# Итоги

- ☐ Не используйте интеграцию через общую базу данных
- ☐ Предпочитайте оркестрацию хореографии
- ☐ REST — отличный выбор для request/response интеграций
- ☒ ! Избегайте обратно несовместимых изменений и необходимости версионировать эндпоинты



# Версионирование

Михаил Триполитов  
Банк Открытие, архитектор решений





# Тема занятия

- 1 Способ версионирования контрактов
- 2 Варианты одновременной работы нескольких версий сервиса
- 3 Соккрытие внутренней сложности системы от внешних потребителей



# Версионирование контрактов

- Поддерживайте обратную совместимость контракта
- Поддерживайте документацию в актуальном состоянии
- Сформулируйте стратегию версионирования и придерживайтесь её



# Версионирование: SemVer

MAJOR.MINOR.PATCH

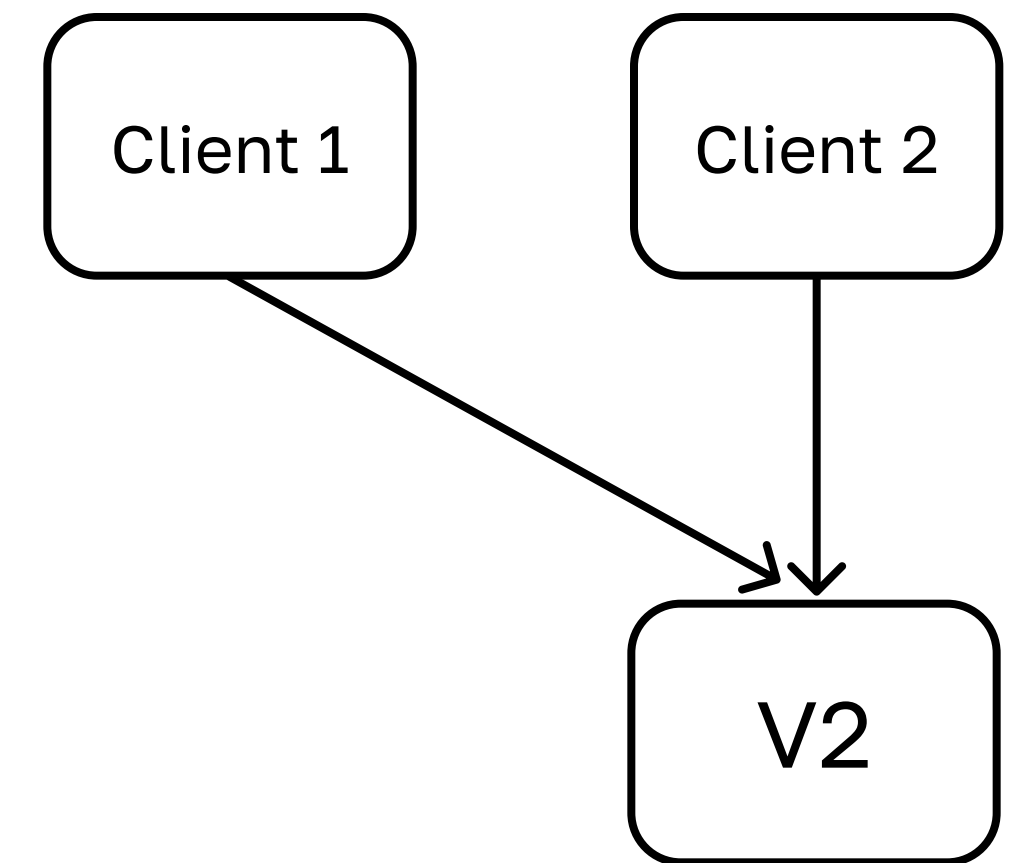
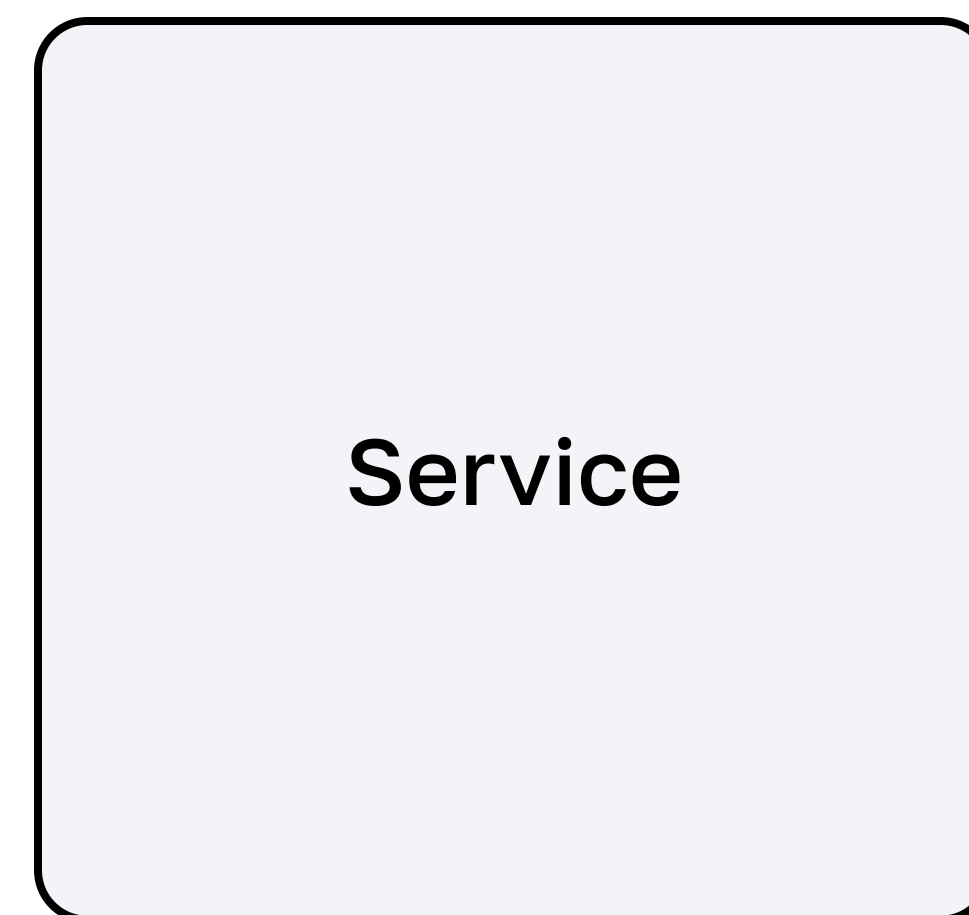
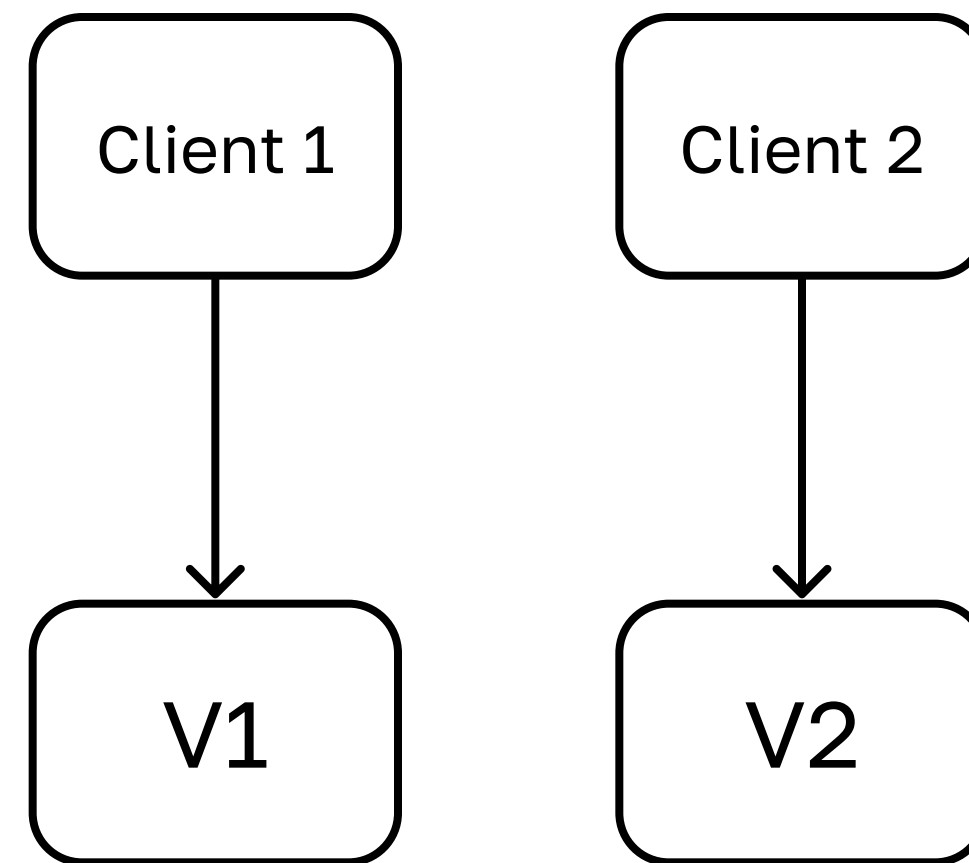
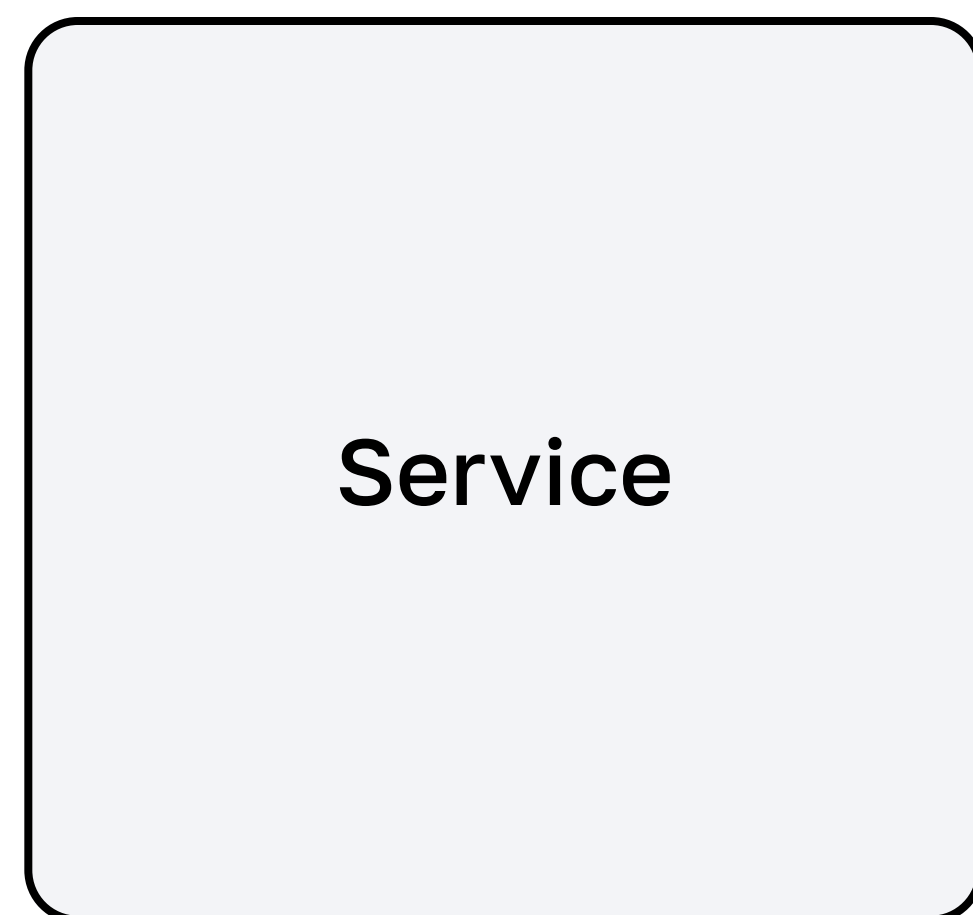
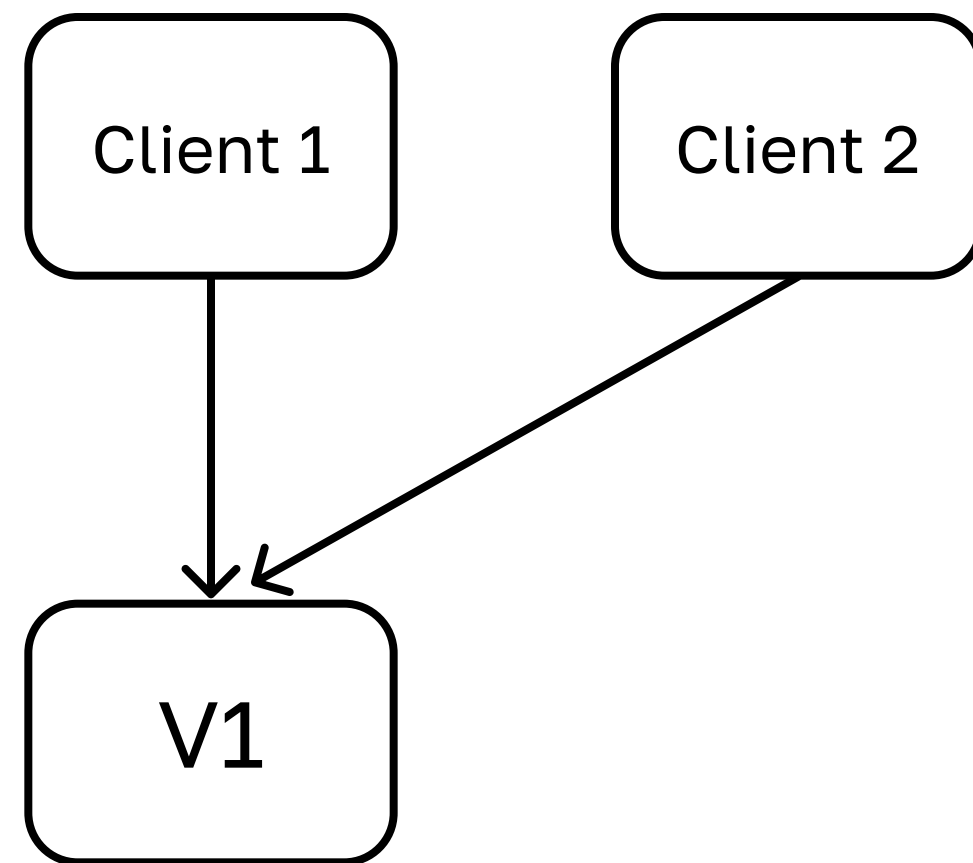
Обратно несовместимые  
изменения

Добавлена новая  
функциональность,  
изменения обратно  
совместимы

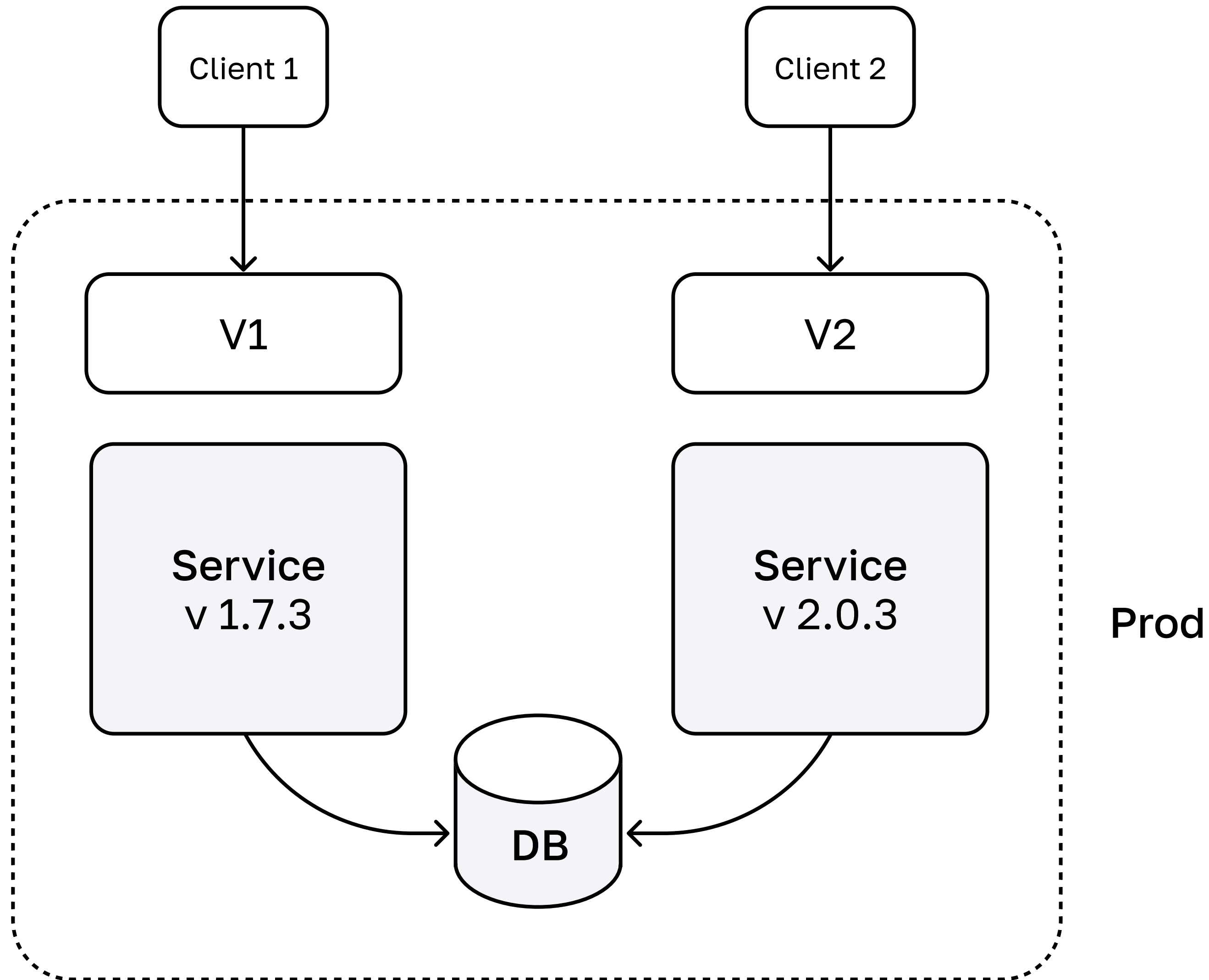
Исправлены ошибки  
в существующей  
функциональности



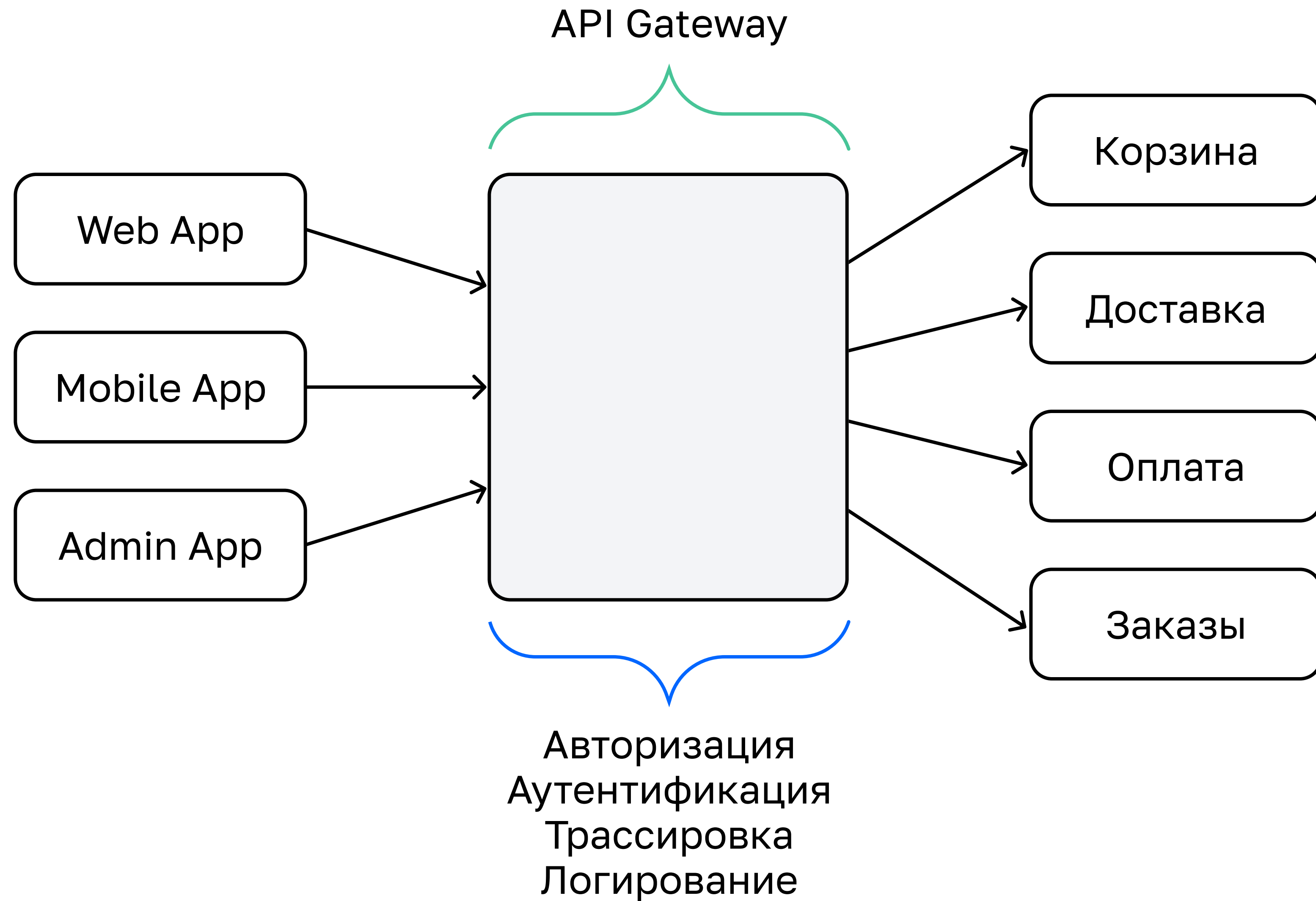
# Версионирование: версии эндпоинтов



# Версионирование: версии сервисов

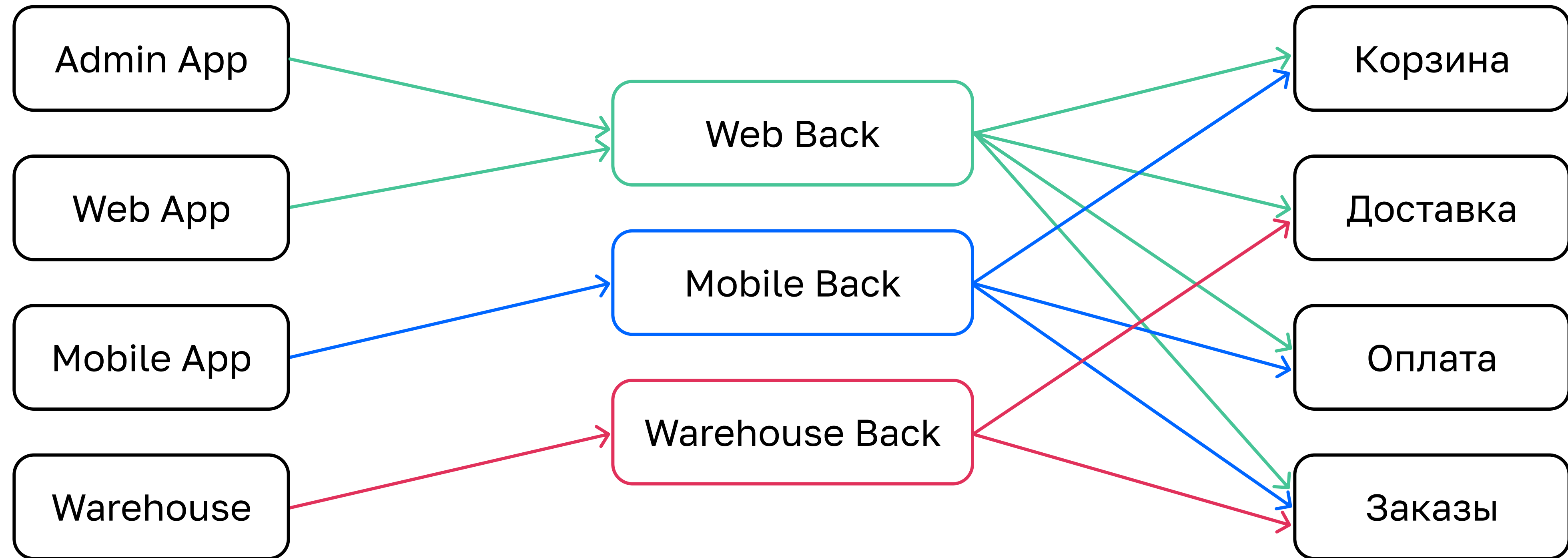


# API Gateway





# Backend for frontend



# Итоги

- ! Избегайте обратно несовместимых изменений и необходимости версионировать эндпоинты
- Применяйте API Gateway или BFF, чтобы скрыть внутреннюю сложность системы от внешних потребителей





# Итоги

- Избегайте обратнo несовместимых изменений и необходимости версионировать эндпоинты
- ! Применяйте API Gateway или BFF, чтобы скрыть внутреннюю сложность системы от внешних потребителей



# Двенадцать факторов

Михаил Триполитов  
Банк Открытие, архитектор решений



# Тема занятия

- 1 Факторы создания и эксплуатации приложения

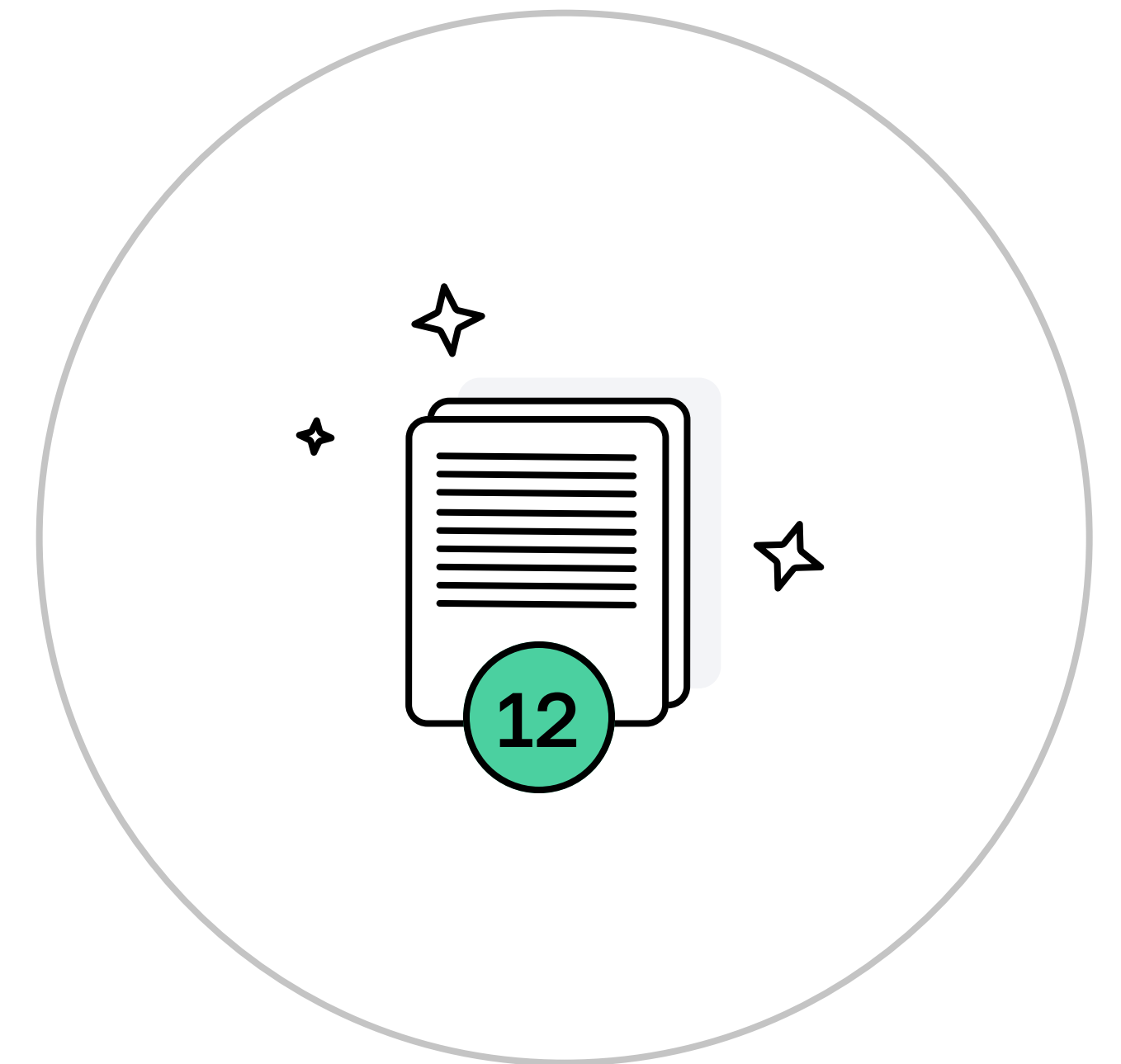




# 12 факторов

12 практик, которые описывают приложения, готовые:

- к горизонтальному масштабированию
- к непрерывному развёртыванию
- к современным облачным хостингам

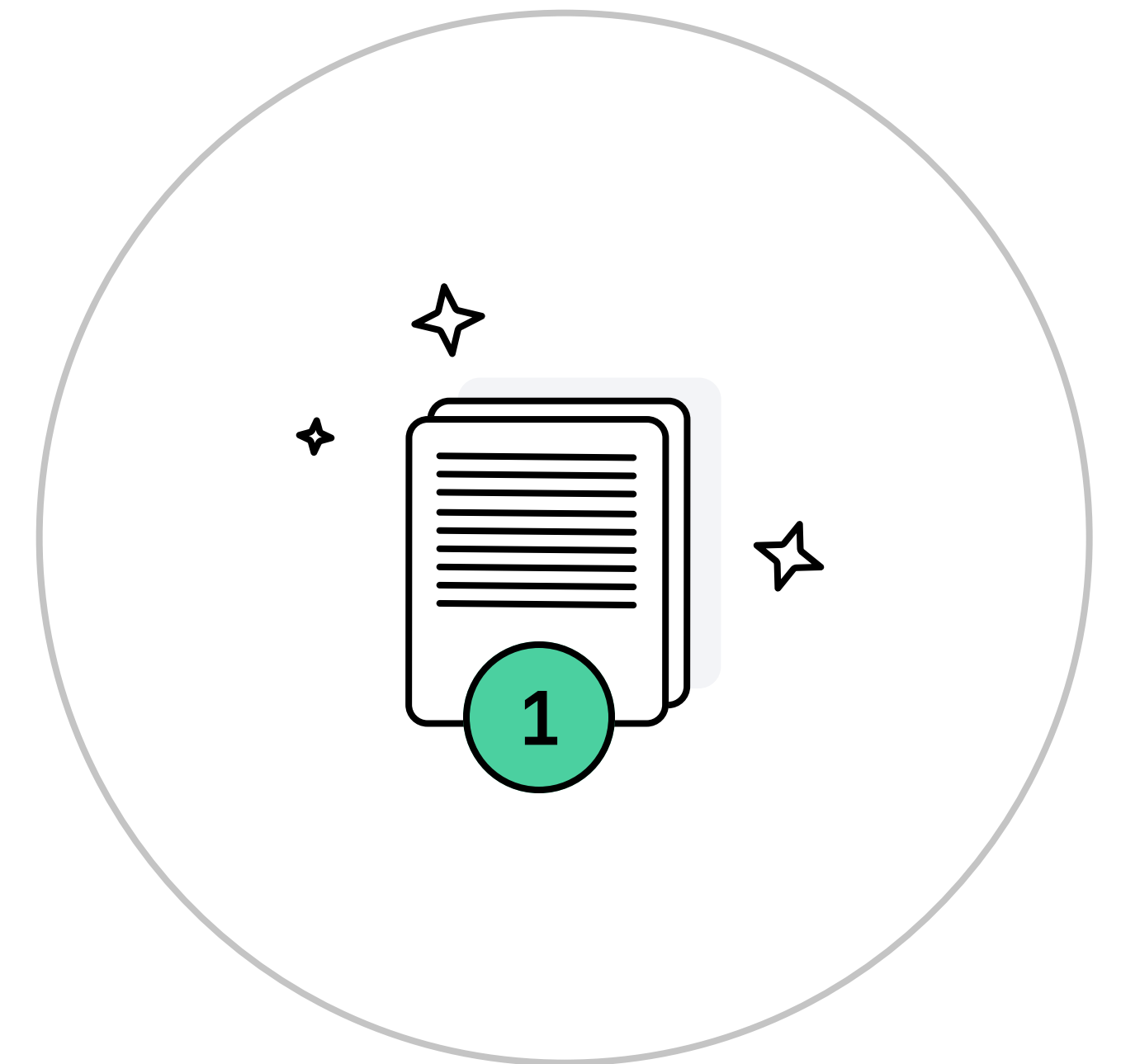


# Фактор 1

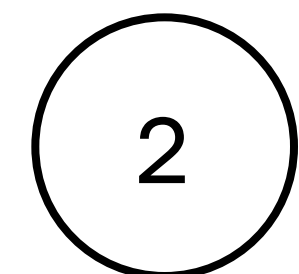
1

## Кодовая база

Одна кодовая база, отслеживаемая в системе контроля версий, — множество развёртываний

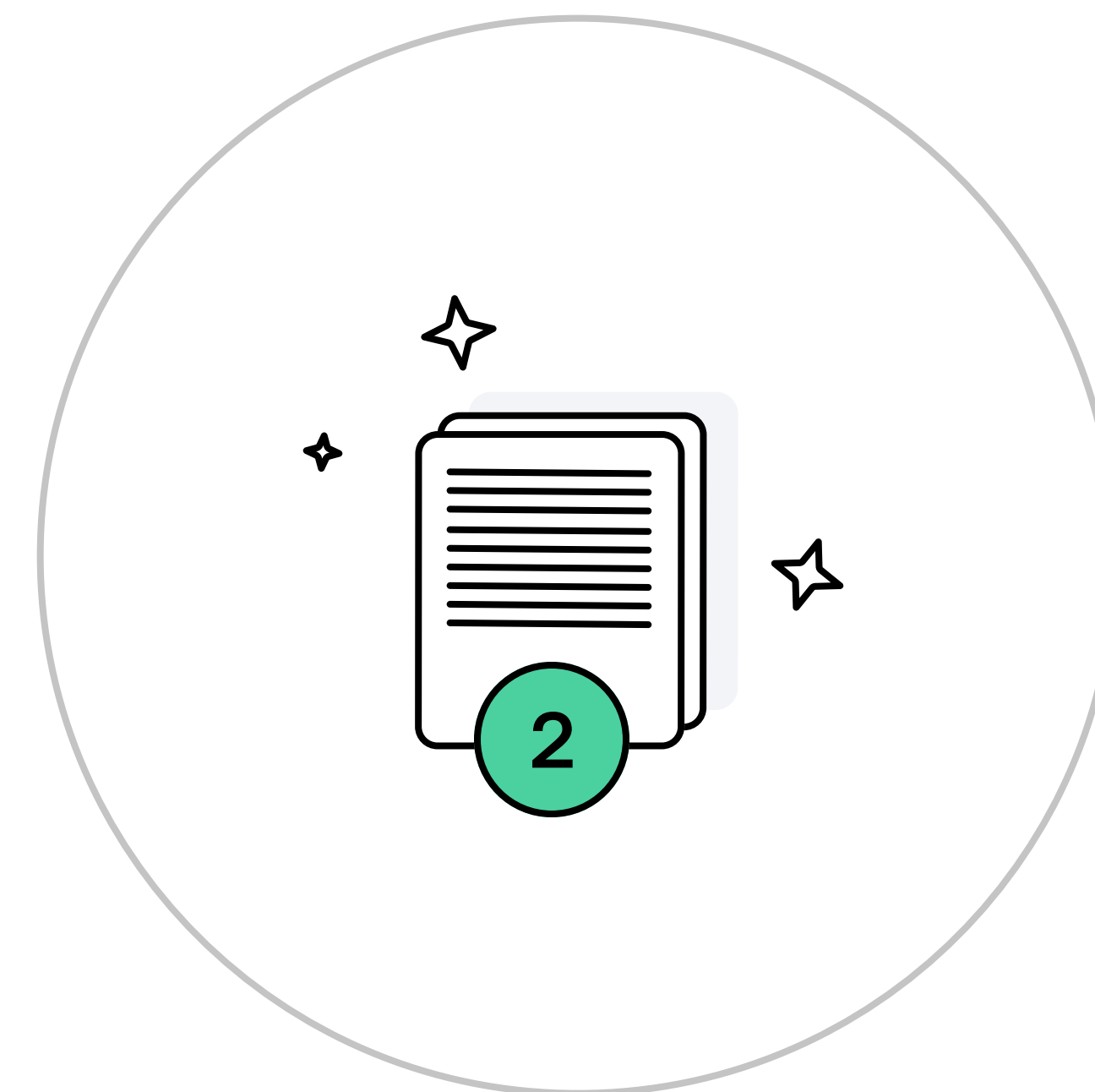


# Фактор 2



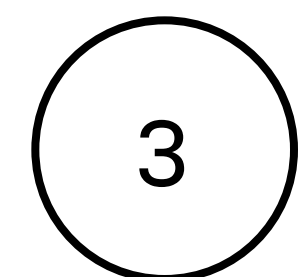
## Зависимости

Явно объявляйте и изолируйте зависимости



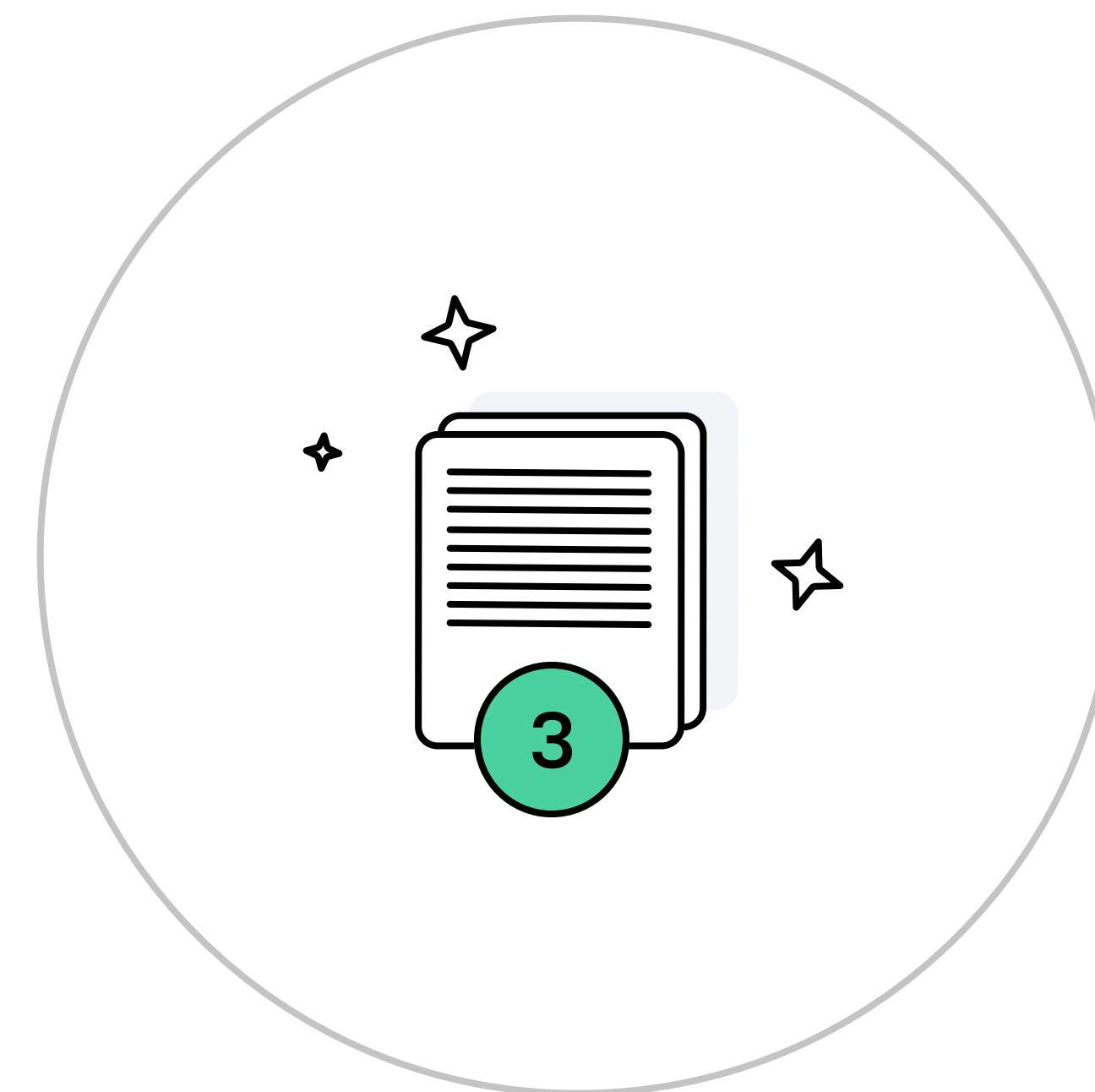


# Фактор 3



## Конфигурация

Сохраняйте конфигурацию в среде выполнения

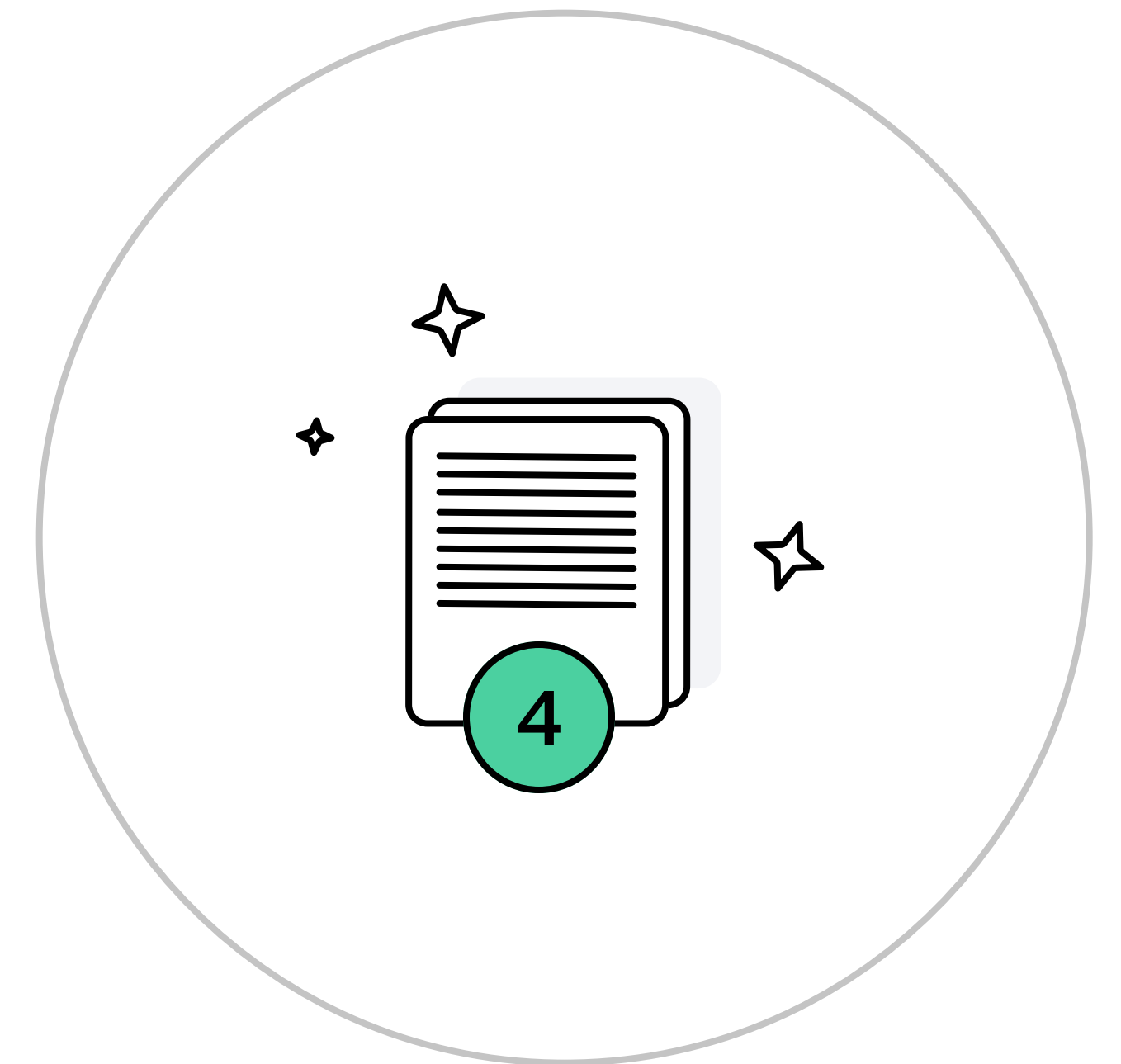


# Фактор 4

4

## Сторонние службы (backing services)

Считайте сторонние службы (backing services) подключаемыми ресурсами



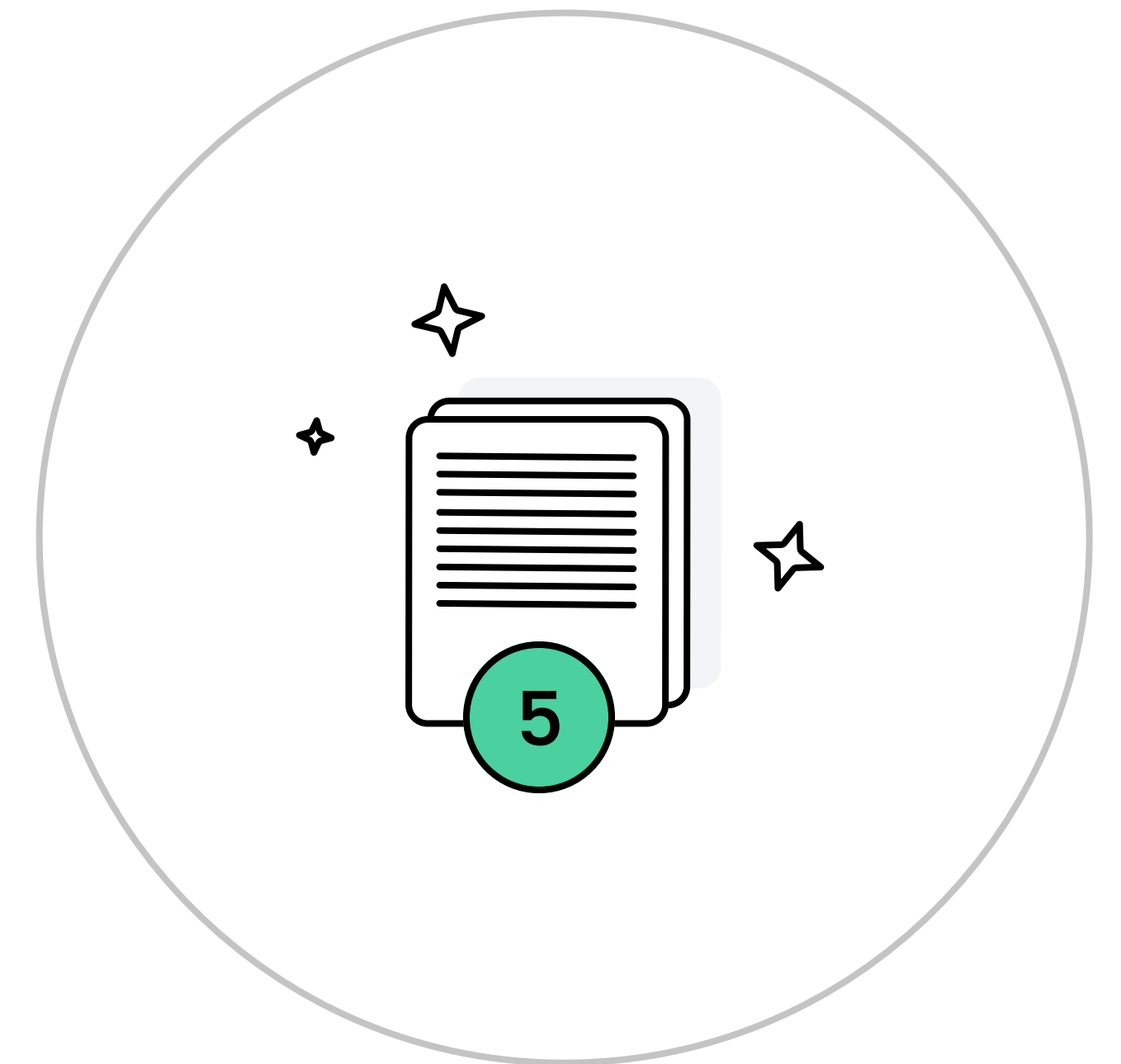


# Фактор 5

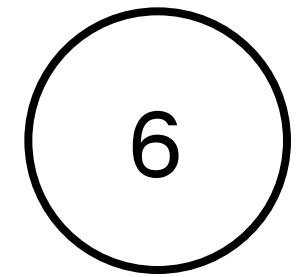
5

## Сборка, релиз, выполнение

Строго разделяйте стадии сборки и выполнения

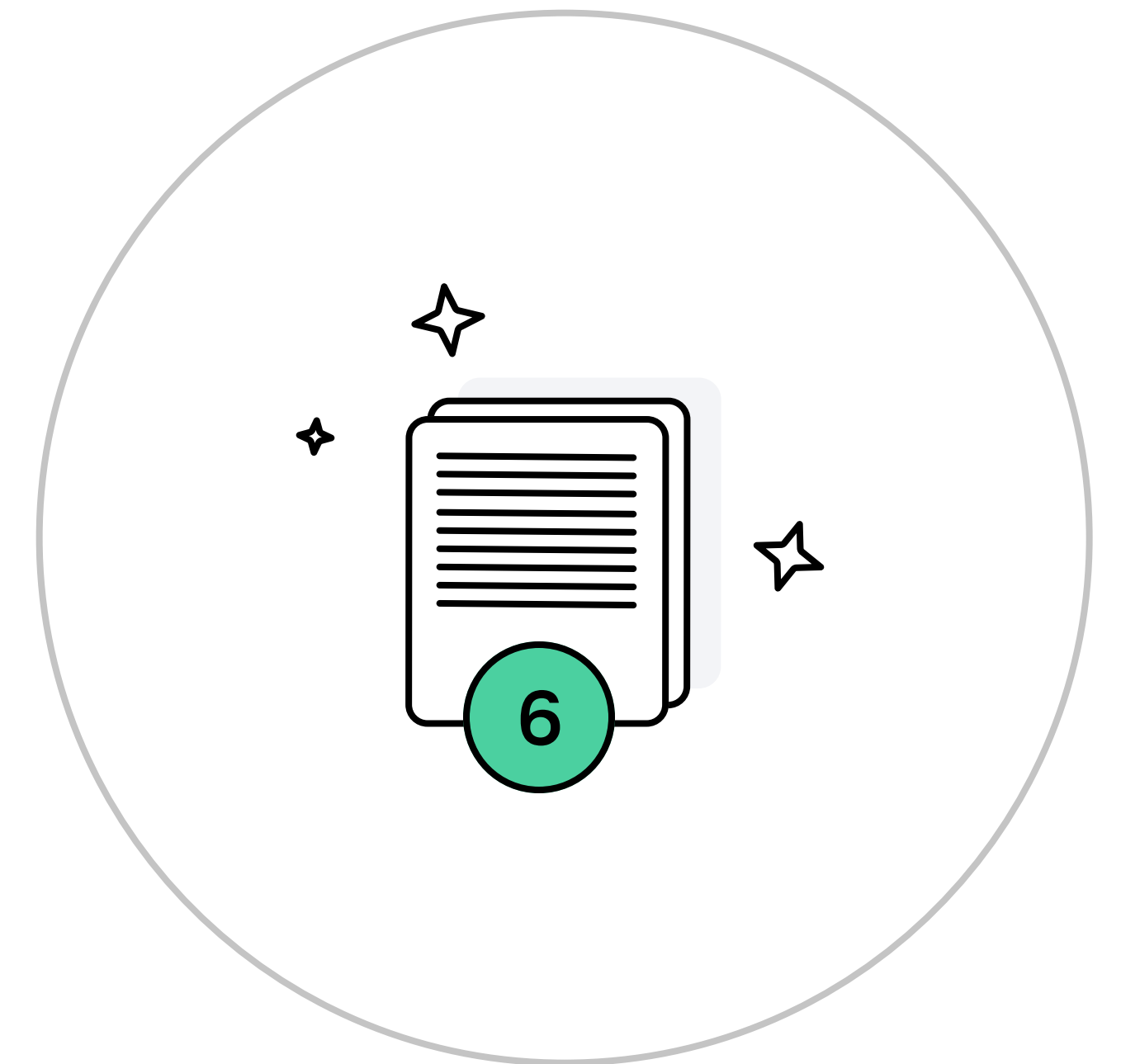


# Фактор 6



## Процессы

Запускайте приложение как один или несколько процессов, не сохраняющих внутреннее состояние (stateless)

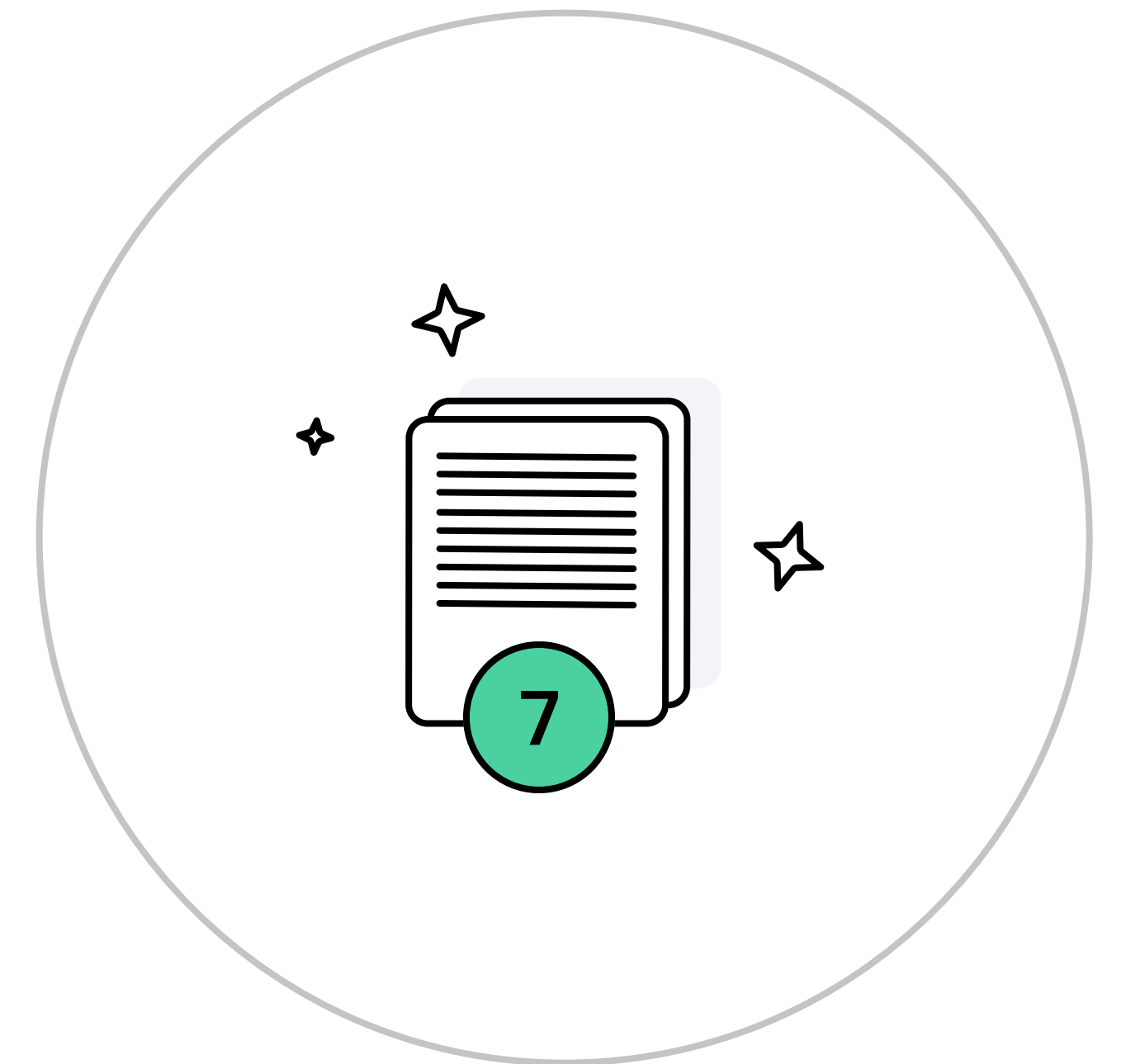


# Фактор 7

7

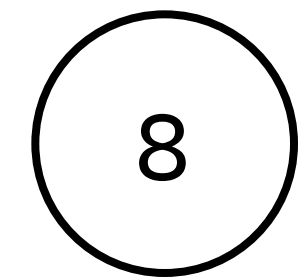
## Привязка портов (Port binding)

Экспортируйте сервисы через привязку портов



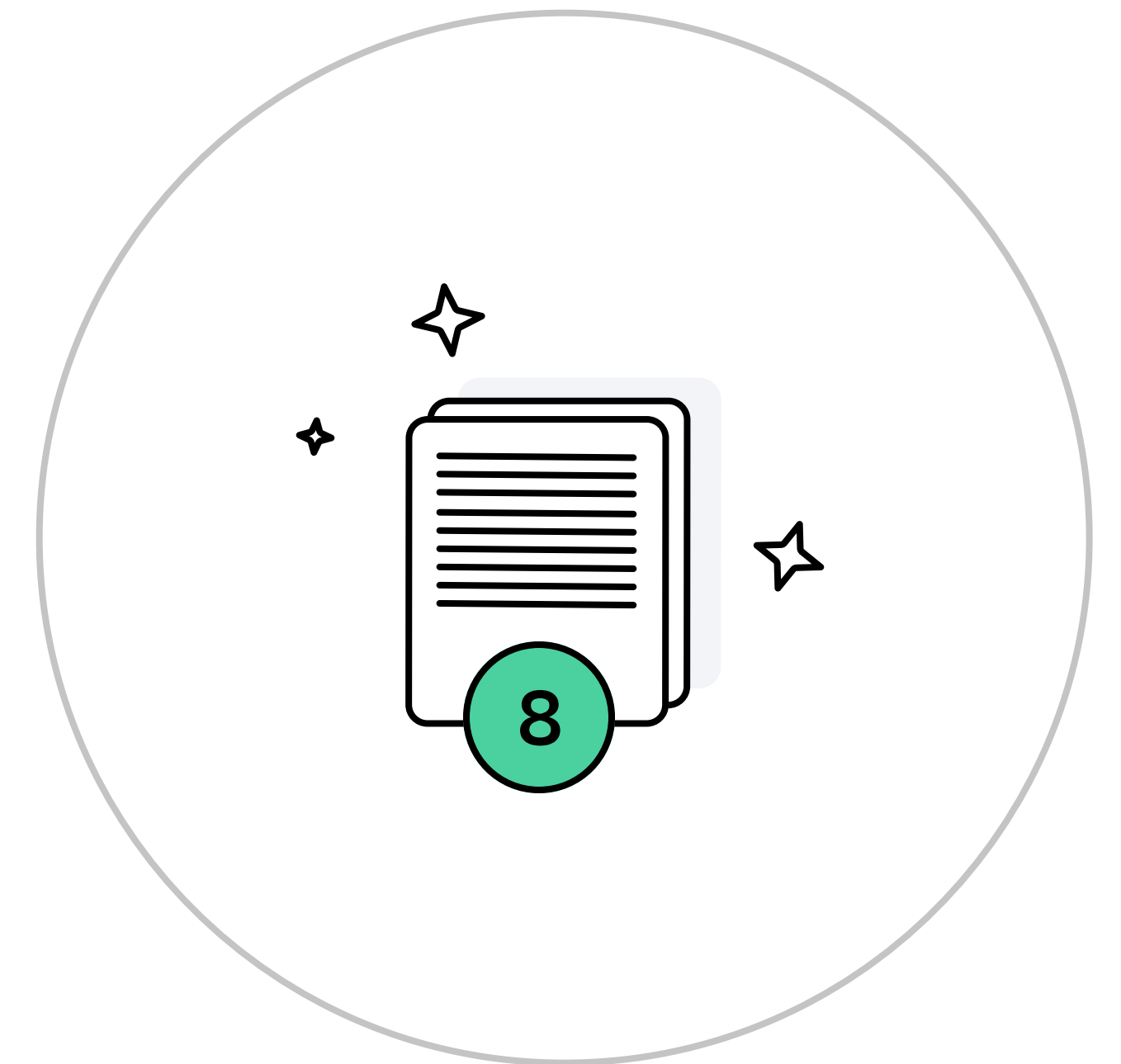


# Фактор 8



## Параллелизм

Масштабируйте приложение с помощью процессов

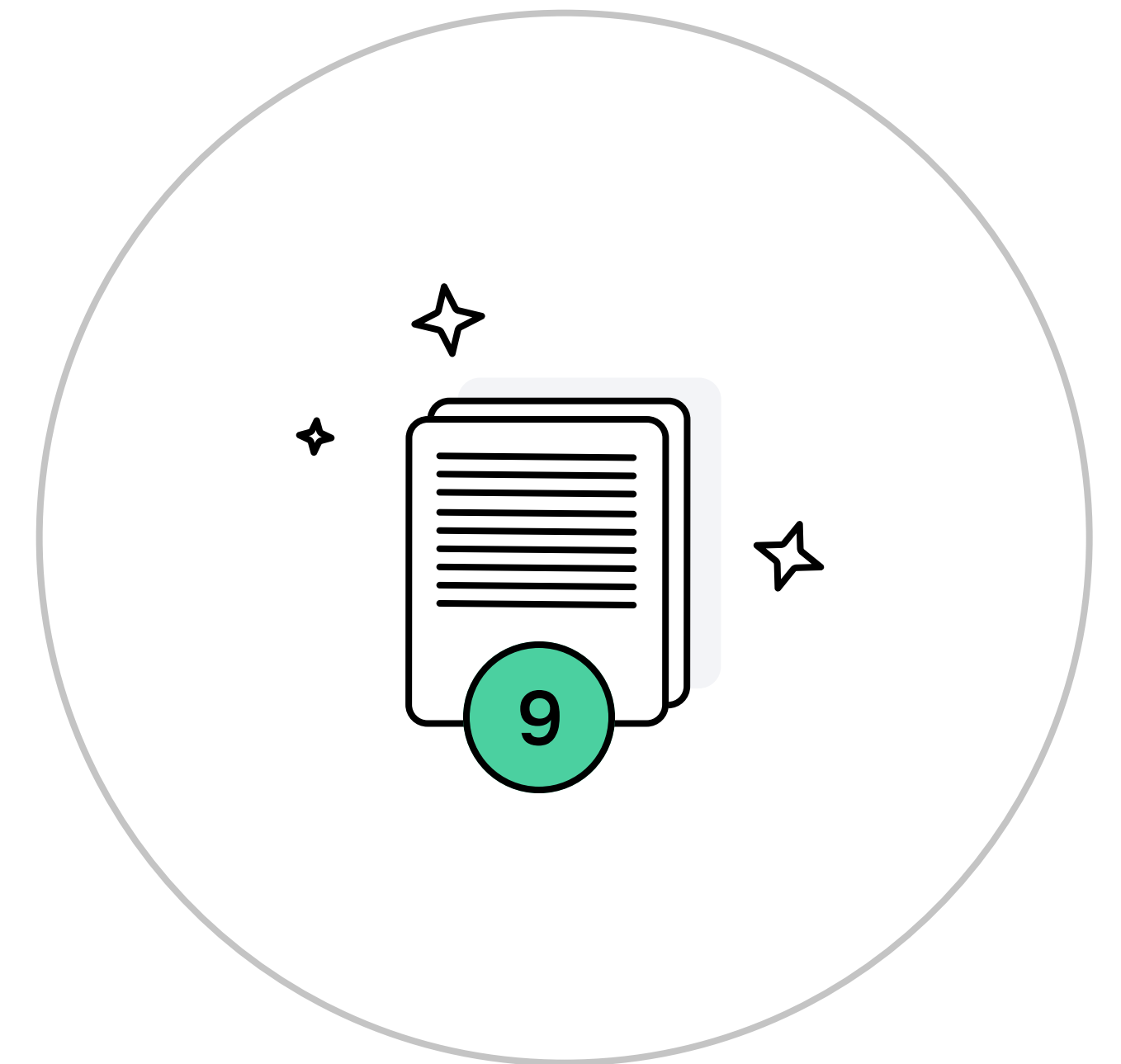


# Фактор 9

9

## Утилизируемость (Disposability)

Максимизируйте надёжность с помощью быстрого запуска и корректного завершения работы

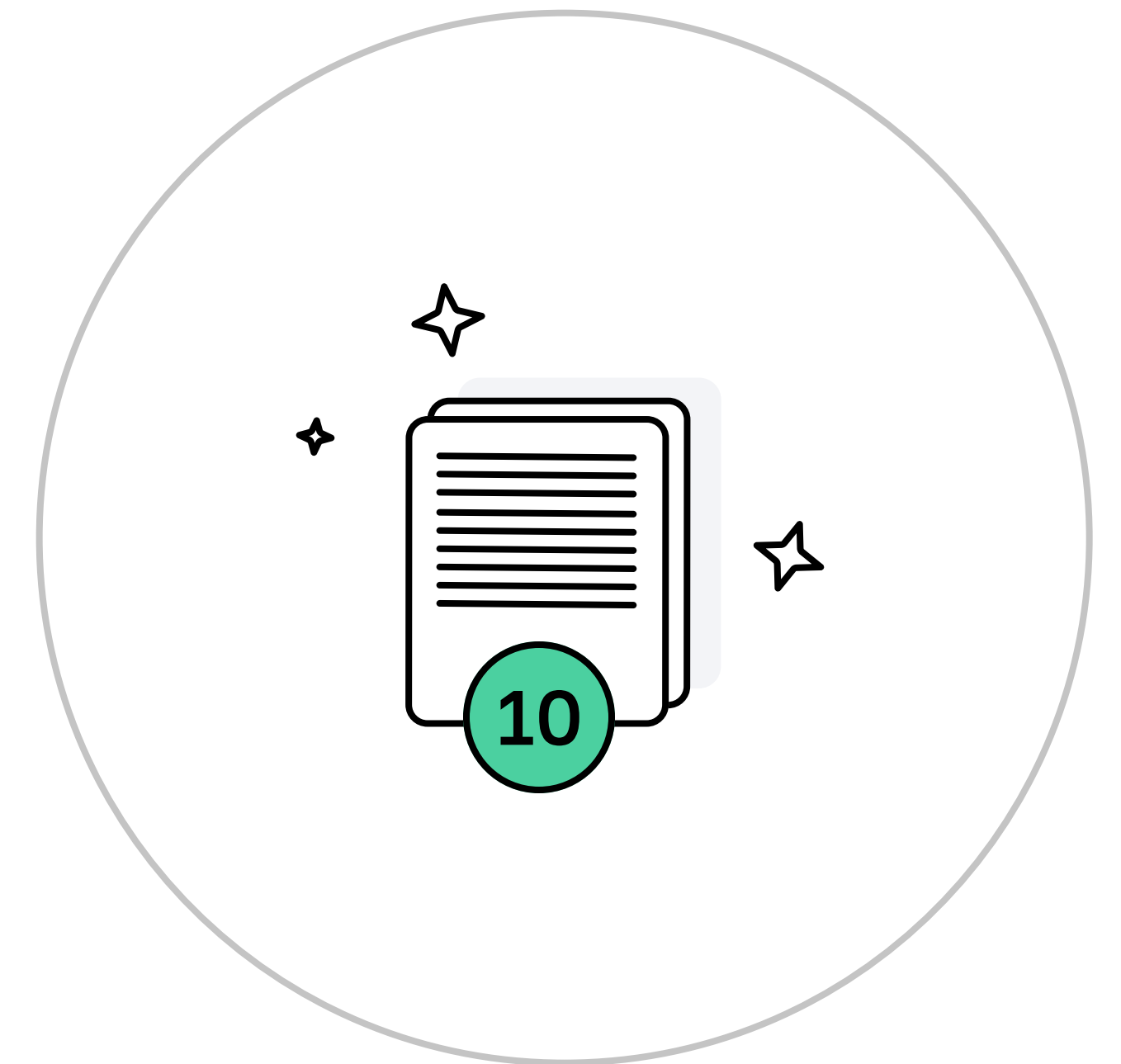


# Фактор 10

10

## Паритет разработки, работы приложения

Держите окружения разработки, промежуточного развёртывания (staging) и рабочего развёртывания (production) максимально похожими





# Фактор 11

11

## Журналирование (Logs)

Рассматривайте журнал как поток событий

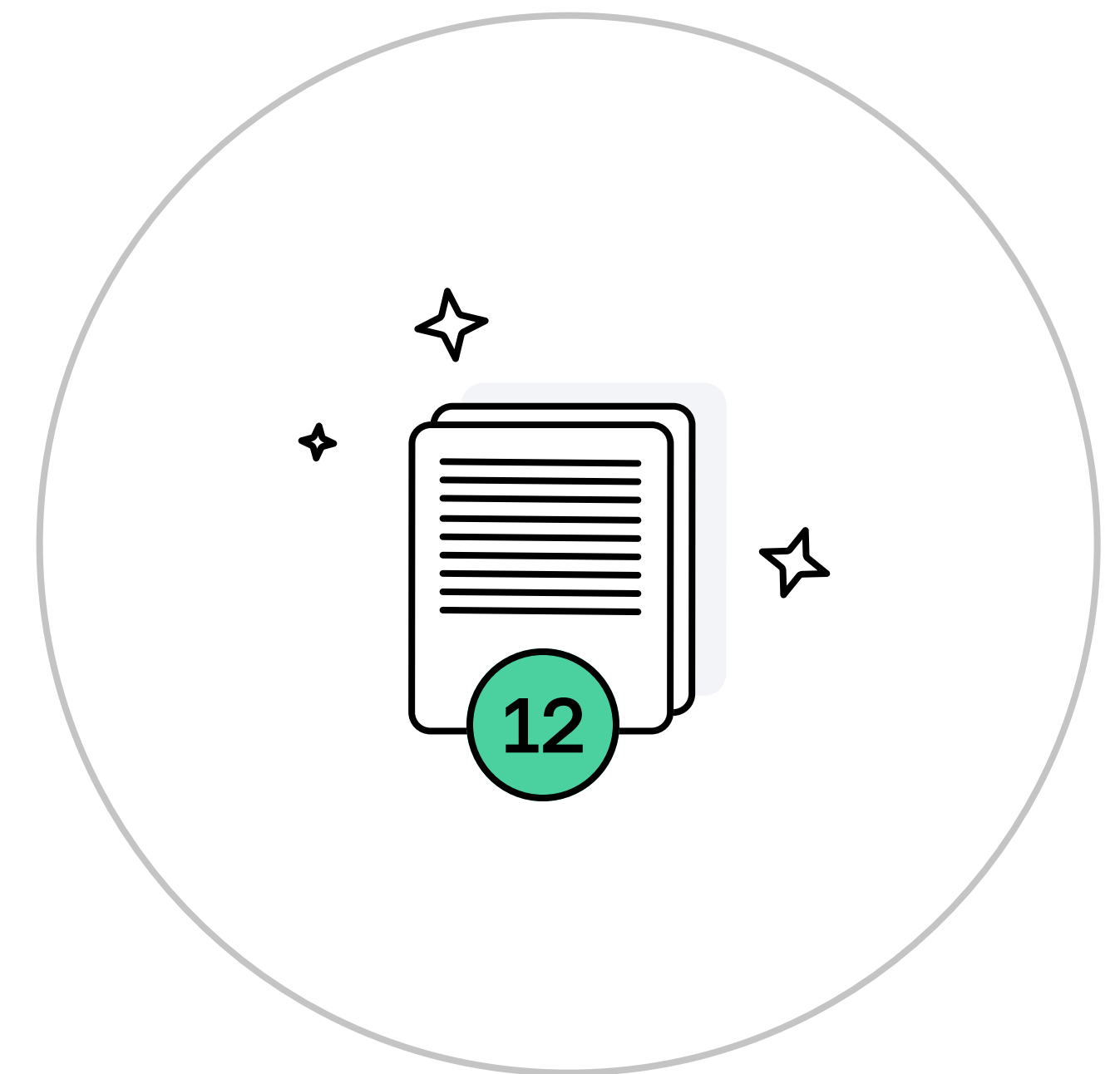


# Фактор 12

12

## Задачи администрирования

Выполняйте задачи администрирования, управления с помощью разовых процессов





# Итоги

- ! 12 факторов — это практики, которыми стоит руководствоваться при создании и эксплуатации приложений, состоящих из набора сервисов
- Практики 12 факторов можно применять к сервисам, написанным на любом языке программирования, любом технологическом стеке
- Применение 12 факторов позволяет создавать сервисы, готовые к горизонтальному масштабированию, непрерывному развёртыванию в современных облачных хостингах



# Итоги

- 12 факторов — это практики, которыми стоит руководствоваться при создании и эксплуатации приложений, состоящих из набора сервисов
- ! Практики 12 факторов можно применять к сервисам, написанным на любом языке программирования, любом технологическом стеке
- Применение 12 факторов позволяет создавать сервисы, готовые к горизонтальному масштабированию, непрерывному развёртыванию в современных облачных хостингах



# Итоги

- 12 факторов — это практики, которыми стоит руководствоваться при создании и эксплуатации приложений, состоящих из набора сервисов
- Практики 12 факторов можно применять к сервисам, написанным на любом языке программирования, любом технологическом стеке
- ! Применение 12 факторов позволяет создавать сервисы, готовые к горизонтальному масштабированию, непрерывному развёртыванию в современных облачных хостингах

